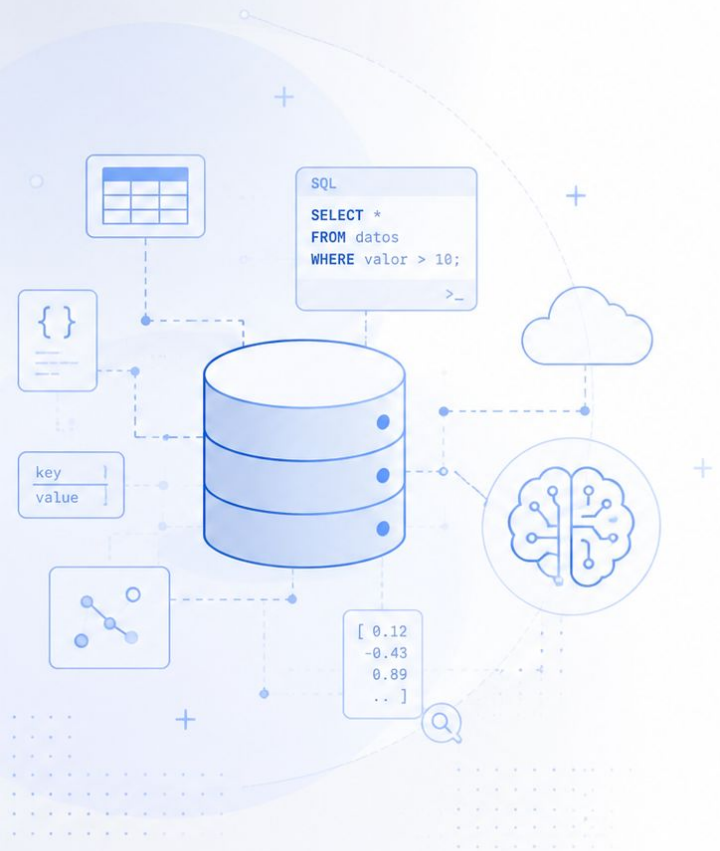




Bases de Datos para Inteligencia Artificial

Especialización en Inteligencia Artificial

Docente:

Lacheski, Martín Anibal

Clase 6:

Bases de Datos Vectoriales,
Concurrencia y Escalabilidad

Bases de Datos para IA

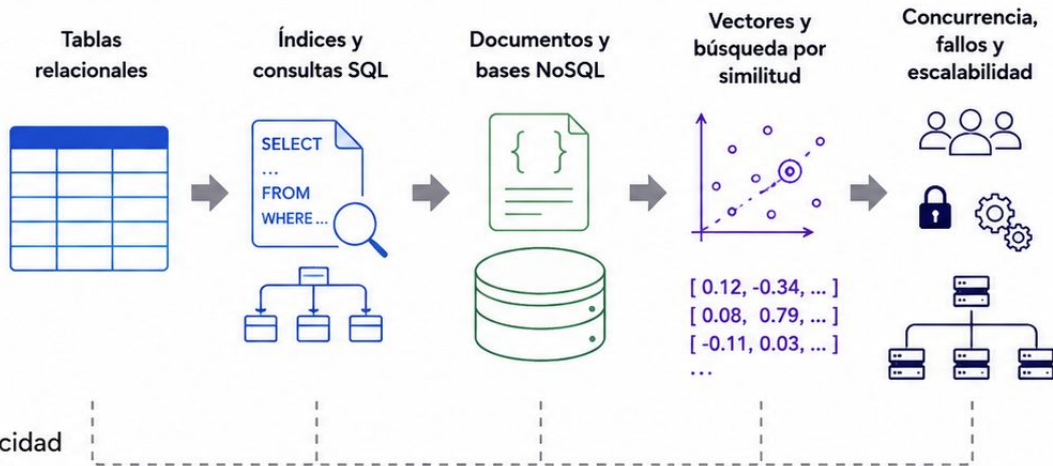
Vectores, concurrencia y distribución

En clases anteriores trabajamos con:

- modelado relacional;
- consultas SQL;
- índices y planes de ejecución;
- documentos y bases NoSQL;
- transacciones y propiedades ACID;
- escalabilidad vertical y horizontal;
- arquitecturas de datos distribuidas.

También presentamos las bases vectoriales como una alternativa para buscar elementos similares.

En esta clase vamos a analizar cómo incorporar esa capacidad dentro de una solución de datos y cómo sostenerla cuando existen múltiples usuarios, fallos y crecimiento de carga.



De estructuras tradicionales a sistemas modernos, distribuidos y confiables



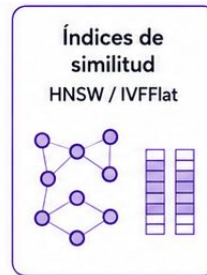
No vamos a estudiar modelos de IA.

Vamos a estudiar cómo una base almacena, consulta y administra sus resultados.

Bases de Datos para IA

Agenda del día

- Bases de datos vectoriales dentro de una arquitectura de datos.
- Modelado de documentos, fragmentos, embeddings y metadatos.
- Almacenamiento vectorial en PostgreSQL con pgvector.
- Consultas por similitud combinadas con SQL.
- Búsqueda exacta e índices HNSW e IVFFlat.
- Rendimiento y planes de ejecución.
- Actualización, versionado y reindexación.
- RAG como caso de uso de una base vectorial.
- Transacciones y control de concurrencia.
- Recuperabilidad y tolerancia a fallos.
- Replicación, particionamiento y escalabilidad.
- Práctica guiada con PostgreSQL y pgvector.



De la búsqueda vectorial a sistemas de datos **confiables y escalables**.



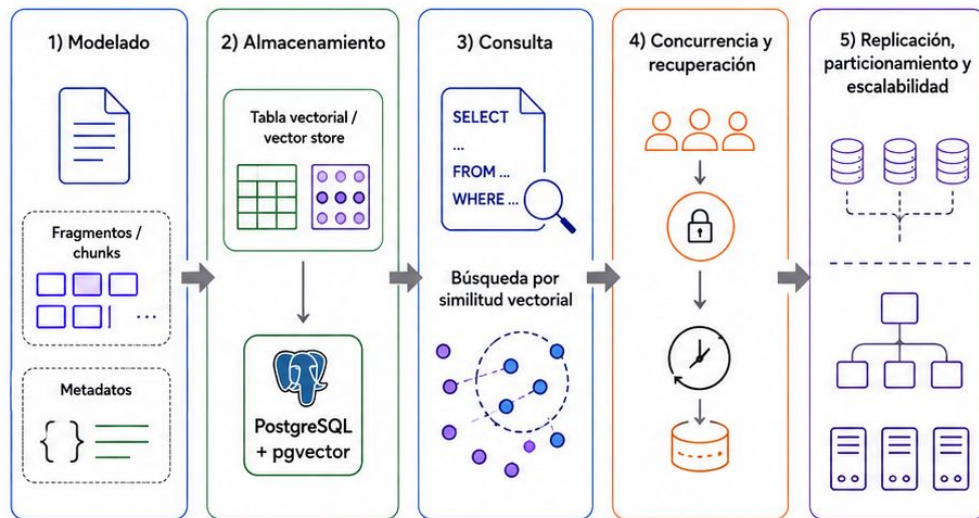
La clase une dos ejes:
búsqueda vectorial y administración confiable del sistema.

Objetivos de la clase

Incorporar búsqueda vectorial en sistemas concurrentes y escalables

Al finalizar la clase, deberíamos poder:

- modelar documentos, fragmentos, embeddings y metadatos;
- almacenar vectores en PostgreSQL mediante pgvector;
- combinar filtros SQL con búsqueda por similitud;
- diferenciar búsqueda exacta de búsqueda indexada;
- comprender el propósito de HNSW e IVFFlat;
- analizar planes de ejecución de consultas vectoriales;
- diseñar procesos de actualización y reindexación;
- reconocer problemas de concurrencia;
- diferenciar recuperación, backup y replicación;
- distinguir replicación de particionamiento;
- justificar decisiones de escalabilidad.



Del modelado vectorial a la operación confiable y escalable



El objetivo no es comprender cómo se entrena un embedding, sino cómo se administra dentro de una base de datos.

Búsqueda Vectorial

como Problema de Bases de Datos



Nuevas consultas sobre datos existentes.

Consultas vectoriales

Buscar por cercanía además de filtrar por valores

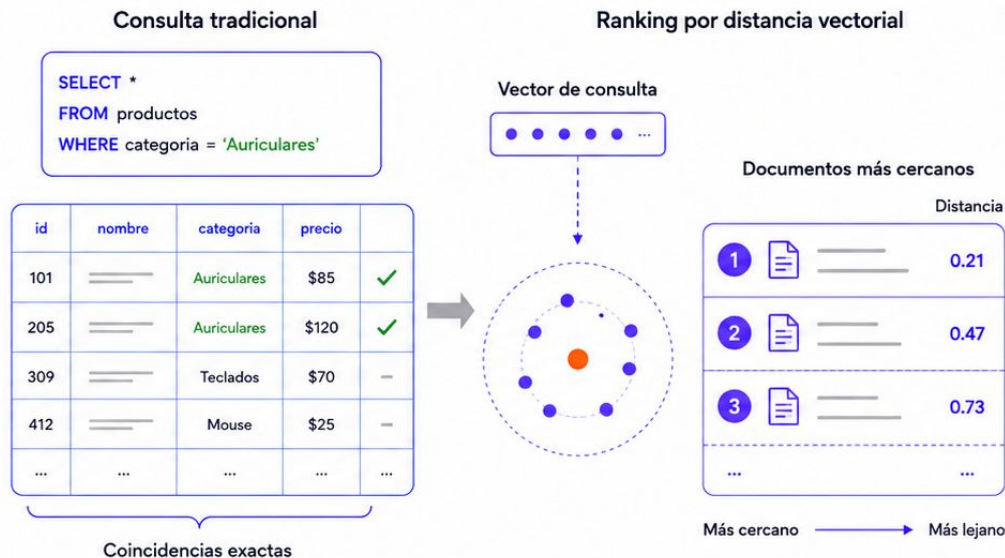
Las bases tradicionales permiten consultar mediante:

- igualdad;
- rangos;
- condiciones;
- relaciones;
- agregaciones;
- coincidencias textuales.

Una consulta vectorial agrega otra posibilidad:

- ordenar registros según la distancia entre dos vectores.

El resultado no es una coincidencia exacta, sino un ranking de elementos similares.



La búsqueda vectorial amplía SQL con un nuevo criterio de ordenamiento.

Dato vectorial

El vector no vive aislado

Para que una búsqueda vectorial sea útil, cada registro necesita vincular:

- un identificador;
- el contenido original;
- el embedding;
- metadatos;
- información de origen;
- versión del contenido;
- modelo utilizado;
- fecha de indexación;
- permisos de acceso.

El vector permite comparar, pero los demás campos permiten interpretar, filtrar y administrar el resultado.

 id	101
 contenido	La búsqueda vectorial amplía SQL...
 embedding	
 metadatos	idioma: es, tipo: artículo, tags: [sql, vectorial]
 origen	blog.empresa.com/busqueda-vectorial
 versión	v1.3.0
 modelo	text-embedding-3-large
 fecha de indexación	2025-05-15 10:32:45 UTC
 permisos	lectura: equipo-data, escritura: admin

El registro combina contenido, vector y datos de administración



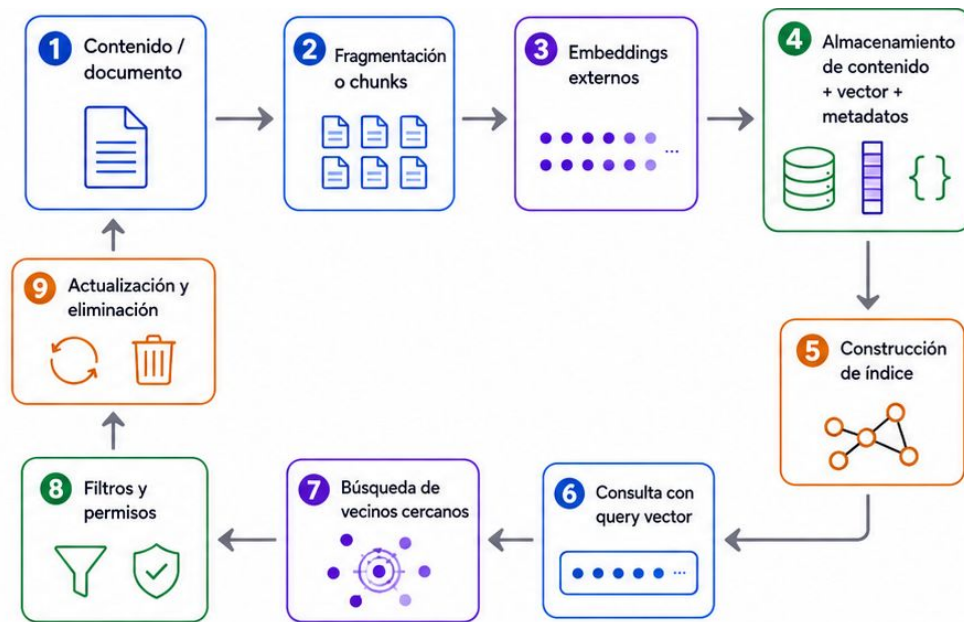
Una solución vectorial necesita modelo de datos, no solamente una colección de vectores.

Flujo de datos vectoriales

Crear, almacenar, consultar y mantener

Desde el punto de vista de la base, el ciclo es:

- recibir o identificar un contenido;
- dividirlo en unidades almacenables;
- recibir los embeddings generados externamente;
- guardar contenido, vector y metadatos;
- construir un índice;
- recibir el vector de una consulta;
- buscar los registros más cercanos;
- aplicar filtros y permisos;
- actualizar o eliminar cuando cambia el contenido.



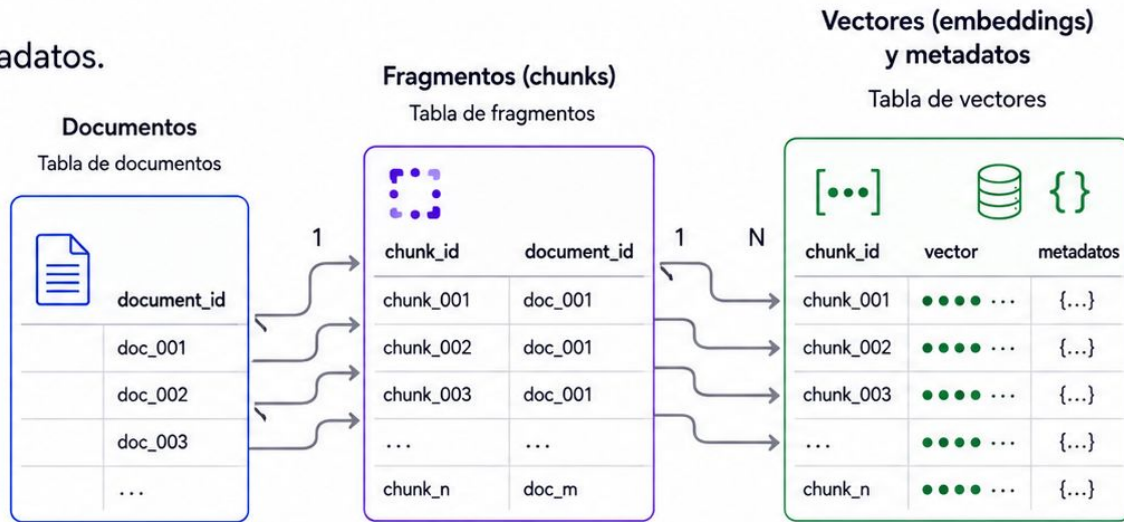
Ciclo de vida de una solución vectorial dentro de la base



La generación del vector es solo una etapa dentro de un ciclo de datos más amplio.

Modelado de Datos Vectoriales

Documentos, fragmentos, vectores y metadatos.



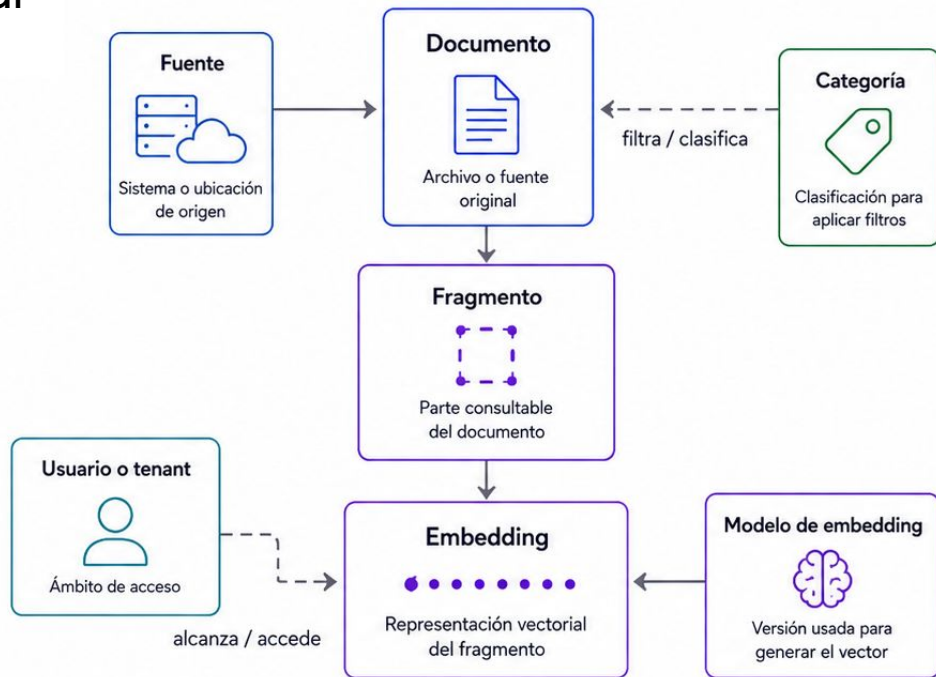
Documentos, fragmentos, vectores y metadatos.

Modelo conceptual

Qué conceptos necesitamos representar

Una solución de búsqueda documental puede incluir:

- **Documento:** archivo o fuente original.
- **Fragmento:** parte consultable del documento.
- **Embedding:** representación vectorial del fragmento.
- **Fuente:** sistema o ubicación de origen.
- **Categoría:** clasificación para aplicar filtros.
- **Modelo de embedding:** versión usada para generar el vector.
- **Usuario o tenant:** ámbito de acceso.



Antes de crear una columna vectorial, debemos identificar qué representa cada registro.

Documento y fragmento

Separar origen de unidad de búsqueda

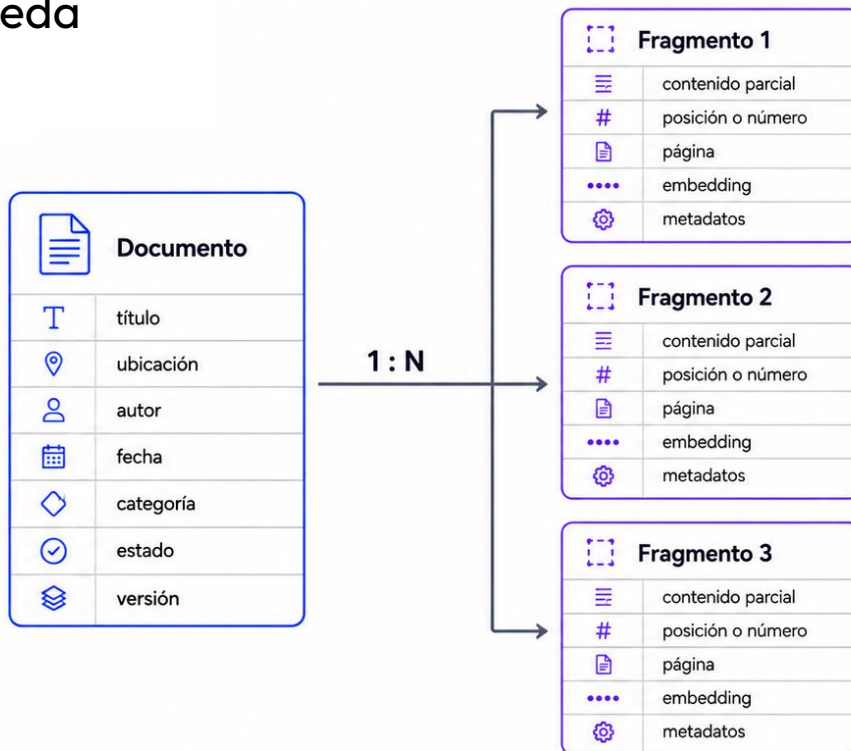
Un documento puede contener muchos fragmentos.

El documento conserva:

- título;
- ubicación;
- autor;
- fecha;
- categoría;
- estado;
- versión general.

El fragmento conserva:

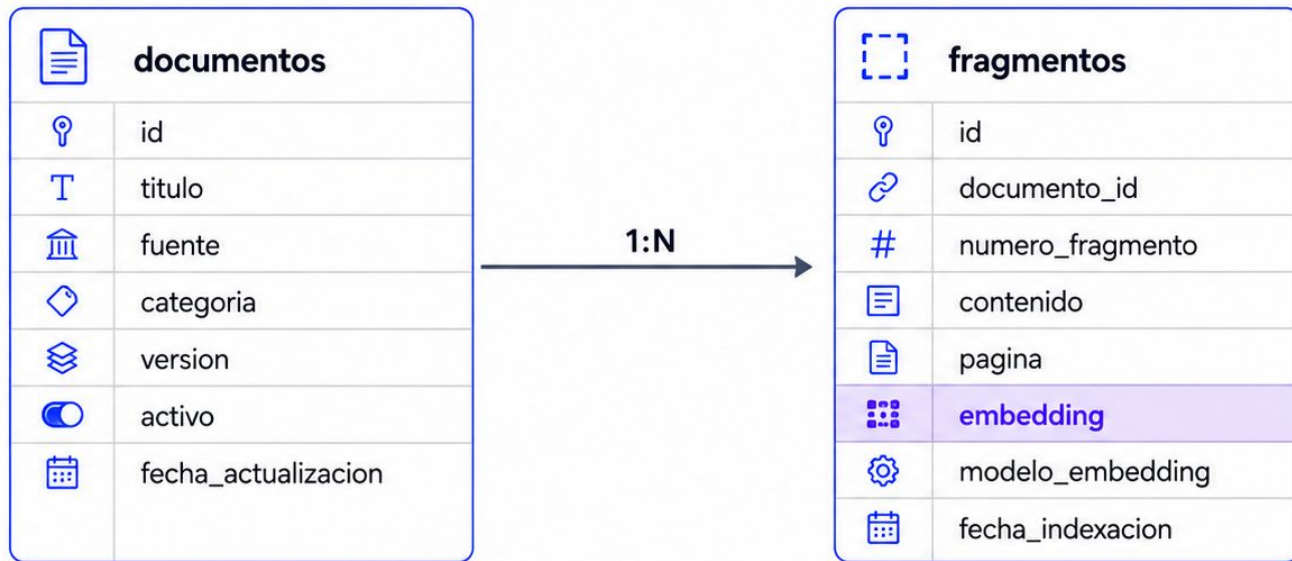
- contenido parcial;
- número o posición;
- página;
- embedding;
- metadatos específicos.



El documento representa la fuente.
El fragmento representa la unidad que se recupera.

Modelo lógico

Tablas para documentos y fragmentos



documentos 1:N fragmentos

La columna `documento_id` en fragmentos conecta cada fragmento con su documento.



La búsqueda vectorial puede incorporarse sin abandonar el modelo relacional.

Diseño integrado

Contenido y vector en un único registro

Guardar contenido y embedding en la misma tabla puede simplificar:

- inserciones;
- consultas;
- filtros;
- eliminación;
- trazabilidad;
- transacciones.

Ejemplo:

fragmentos

- id
- documento_id
- contenido
- embedding

fragmentos		
	id	1
	documento_id	doc_001
	contenido	"La búsqueda vectorial amplía SQL..."
	embedding	[0.12, -0.34, 0.56, ...]

Contenido y vector conviven en la misma fila



Un diseño simple suele ser suficiente cuando cada fragmento tiene una sola representación.

Diseño separado

Varias representaciones para el mismo fragmento

Una tabla separada puede ser útil cuando:

- existen varios modelos de embeddings;
- se comparan versiones;
- se necesitan distintas dimensiones;
- se generan representaciones para texto e imagen;
- los embeddings se actualizan de manera independiente.

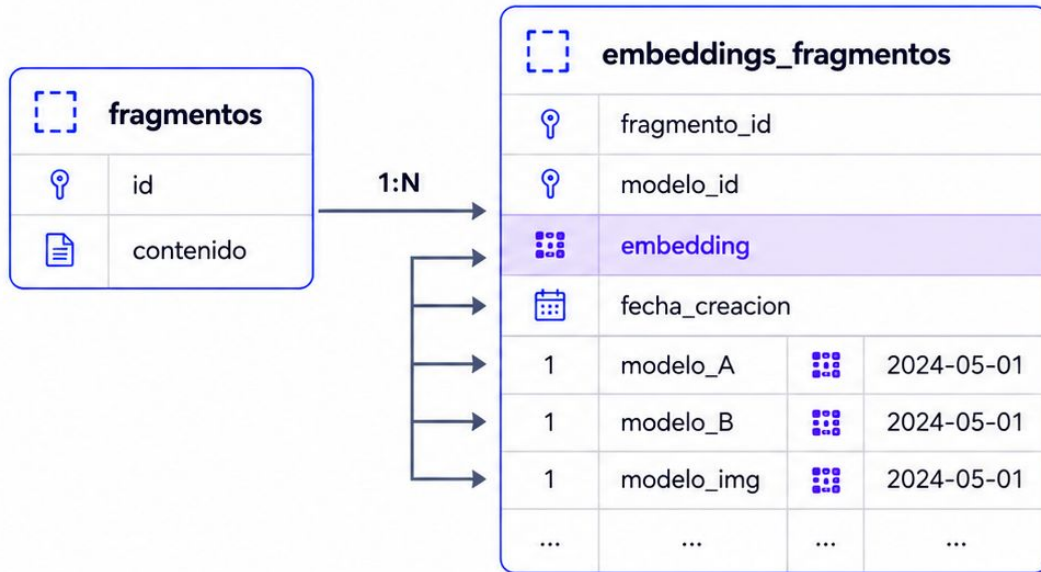
Ejemplo:

fragmentos

- id
- contenido

embeddings_fragmentos

- fragmento_id
- modelo_id
- embedding
- fecha_creacion



Un fragmento puede vincularse con múltiples embeddings



Separar embeddings permite versionar representaciones sin duplicar el contenido.

Dimensionalidad

Una restricción del tipo de dato

Cada modelo produce vectores de una dimensión determinada.

En pgvector, la dimensión puede declararse dentro del tipo:

VECTOR(384)

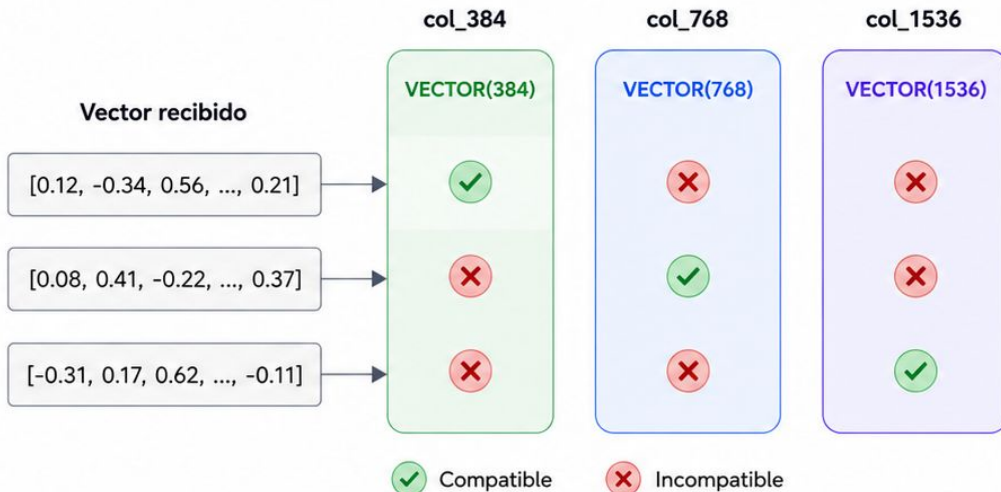
VECTOR(768)

VECTOR(1536)

Esto permite que la base rechace vectores incompatibles.

Cambiar de modelo puede implicar:

- crear otra columna;
- crear otra tabla;
- recalcular embeddings;
- reconstruir índices.



La dimensión no es solo una propiedad del modelo: también es una restricción del esquema.

Metadatos

Columnas estables y atributos variables

Conviene usar **columnas relacionales** para datos que:

- ✓ se filtran frecuentemente;
- ✓ tienen estructura estable;
- ✓ requieren restricciones;
- ✓ participan en relaciones;
- ✓ necesitan índices tradicionales.

JSONB puede utilizarse para:

- ✓ atributos variables;
- ✓ propiedades específicas de una fuente;
- ✓ información complementaria;
- ✓ configuraciones de indexación.

fragmentos	
	id
	documento_id
	categoria
	tenant_id
	fuelle
	activo
	metadata_jsonb <pre>{ "idioma": "es", "pagina": 3, "tags": ["sql", "vectorial"], "ocr": true, "chunk_size": 500 }</pre>

Columnas relacionales

Campos estables, filtrables, con restricciones, relaciones e índices tradicionales.

JSONB complementario

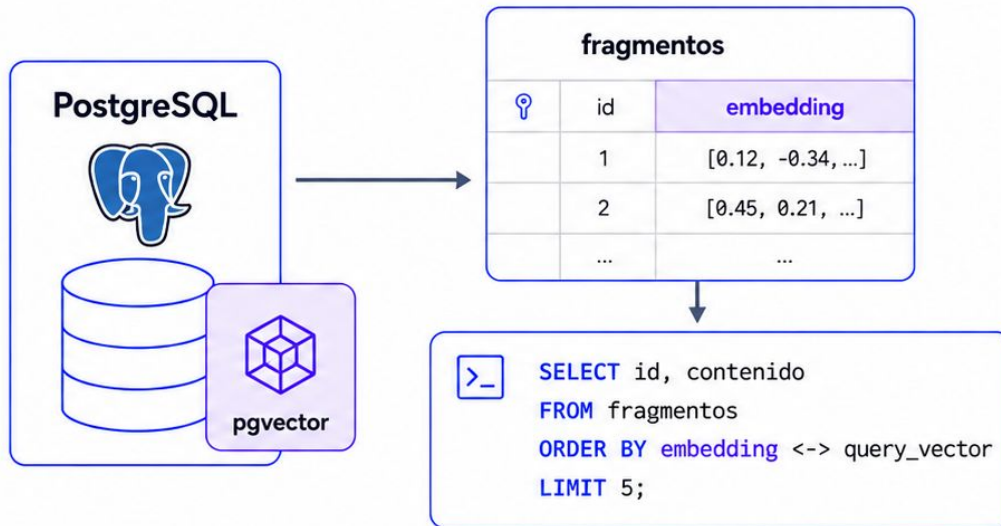
Atributos variables y complementarios para cada fragmento.



Los metadatos importantes para filtrar y proteger datos no deberían ocultarse sin criterio dentro de JSONB.

Almacenamiento y Consultas

Extender PostgreSQL con
datos vectoriales.



Extender PostgreSQL con datos vectoriales.

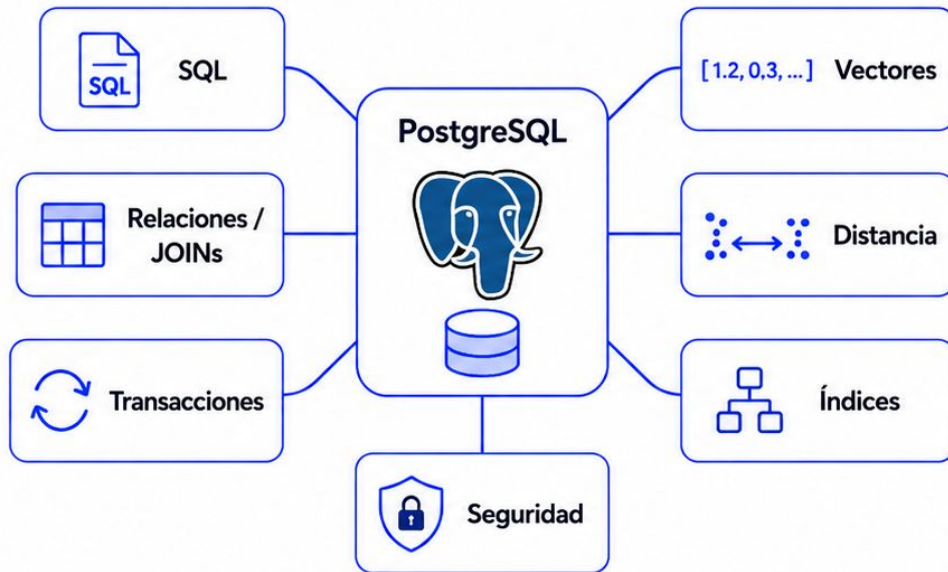
pgvector

Una extensión vectorial para PostgreSQL

pgvector agrega a PostgreSQL:

- un tipo de dato vector;
- operadores de distancia;
- búsqueda exacta;
- índices vectoriales;
- integración con SQL;
- compatibilidad con transacciones;
- combinación con JOINS y filtros;
- seguridad y permisos de PostgreSQL.

No reemplaza PostgreSQL: amplía sus capacidades.



PostgreSQL sigue siendo la base; pgvector suma búsqueda vectorial.



pgvector permite incorporar similitud vectorial dentro de una base relacional existente.

CREATE EXTENSION

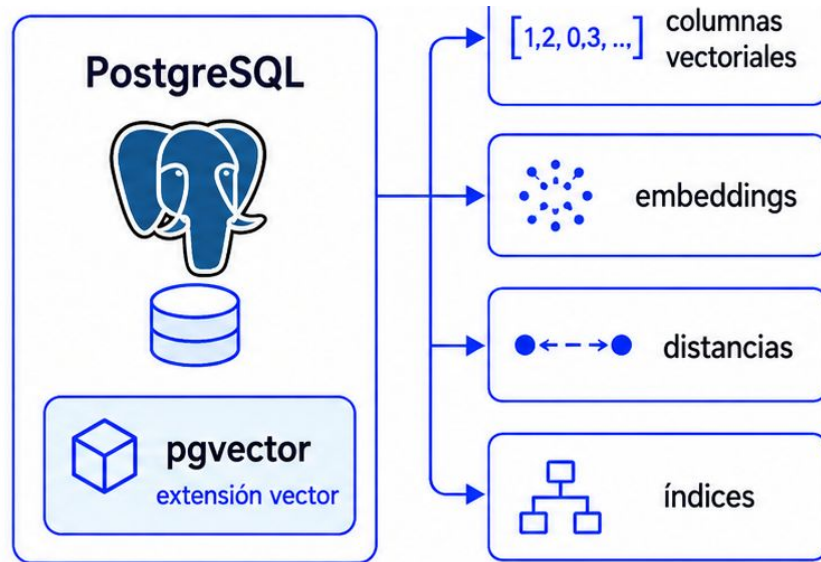
Agregar el tipo vector

```
CREATE EXTENSION IF NOT EXISTS vector;
```

La extensión debe estar instalada en el servidor y habilitada dentro de la base.

Después de habilitarla podemos:

- declarar columnas vectoriales;
- insertar embeddings;
- calcular distancias;
- crear índices especializados.



El soporte vectorial se incorpora como una extensión del motor existente.

CREATE TABLE

Contenido, metadatos y embedding

```
CREATE TABLE fragmentos (  
  id BIGSERIAL PRIMARY KEY,  
  documento_id BIGINT NOT NULL  
    REFERENCES documentos(id),  
  numero_fragmento INTEGER NOT NULL,  
  contenido TEXT NOT NULL,  
  pagina INTEGER,  
  categoria TEXT,  
  embedding VECTOR(384),  
  modelo_embedding TEXT NOT NULL,  
  fecha_indexacion TIMESTAMP  
    DEFAULT CURRENT_TIMESTAMP  
);
```

fragmentos		
	id	PK
	documento_id	FK → documentos(id)
	numero_fragmento	
	contenido	
	pagina	
	categoria	
	embedding	VECTOR(384)
	modelo_embedding	
	fecha_indexacion	

PK, FK, campos tradicionales y VECTOR(384)



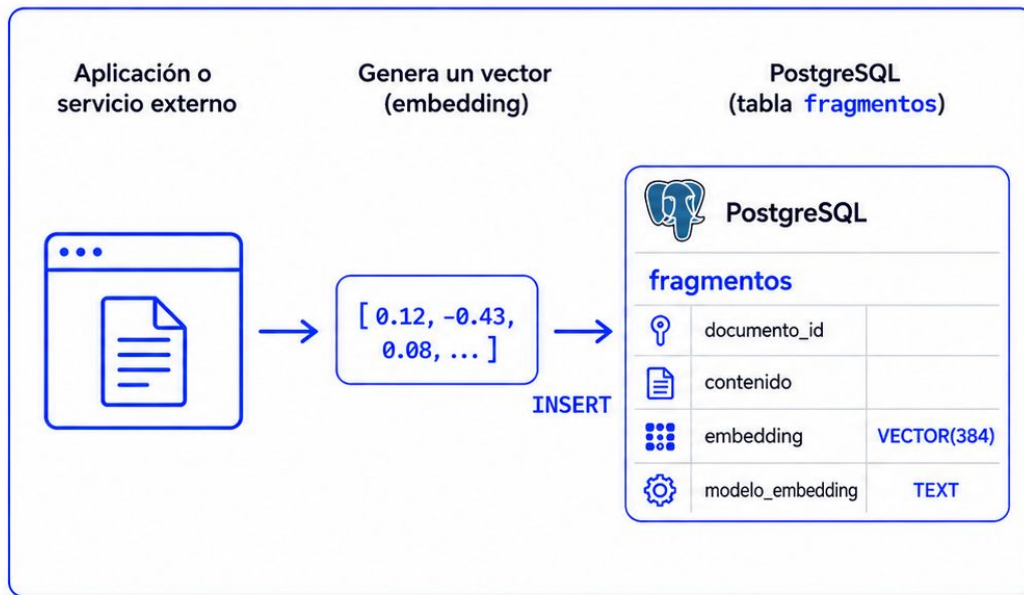
La columna vectorial forma parte de una estructura con claves, contenido y metadatos.

INSERT

Guardar un vector recibido desde la aplicación

```
INSERT INTO fragmentos (  
  documento_id,  
  numero_fragmento,  
  contenido,  
  pagina,  
  categoria,  
  embedding,  
  modelo_embedding  
)  
VALUES (  
  1,  
  1,  
  'Para restablecer la contraseña...',  
  4,  
  'cuentas',  
  '[0.12, -0.43, 0.08, ...]',  
  'modelo_embeddings_v1'  
);
```

El vector normalmente es generado por una aplicación o servicio externo y luego enviado a la base.



La base valida, almacena y consulta el vector; no necesariamente lo genera.

Operadores vectoriales

Expresar similitud dentro de SQL

pgvector ofrece operadores para comparar vectores.

Ejemplos:

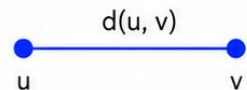
- `<->` distancia euclidiana;
- `<#>` producto interno negativo;
- `<=>` distancia coseno.

La elección debe coincidir con:

- el modelo utilizado;
- la métrica esperada;
- la clase de operador del índice.

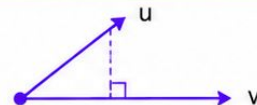
`<->`

distancia euclidiana



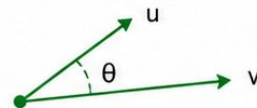
`<#>`

producto interno negativo



`<=>`

distancia coseno



Depende de:

- ✓ modelo
- ✓ métrica
- ✓ clase de operador del índice

La consulta y el índice deben usar una métrica coherente.



La métrica de similitud también forma parte del diseño de la consulta y del índice.

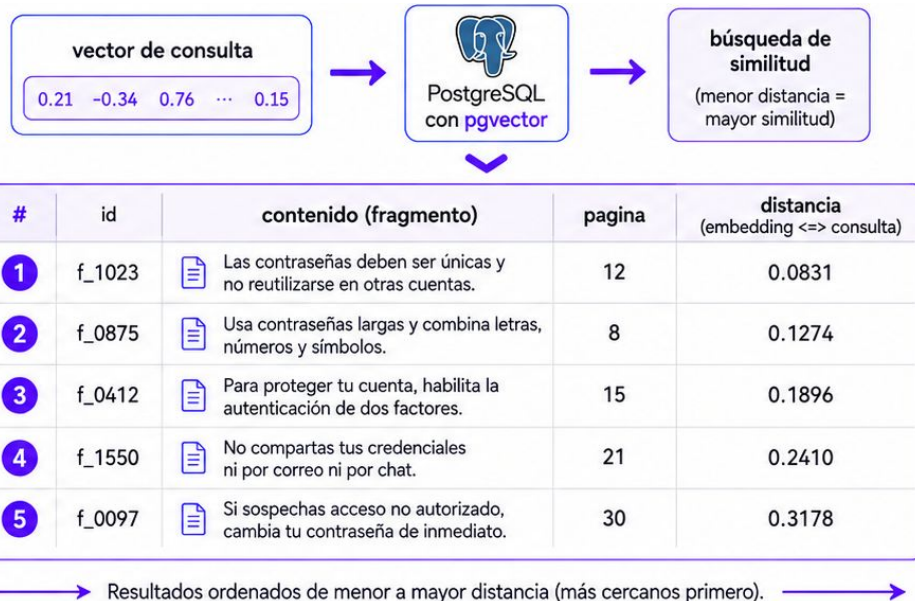
ORDER BY distancia

Recuperar los vecinos más cercanos

```
SELECT
  id,
  contenido,
  pagina,
  embedding <=> :vector_consulta
  AS distancia
FROM fragmentos
ORDER BY embedding <=> :vector_consulta
LIMIT 5;
```

La consulta:

- calcula distancia;
- ordena de menor a mayor;
- limita los resultados;
- conserva contenido y metadatos.



Una búsqueda vectorial es una consulta ordenada por una expresión de distancia.

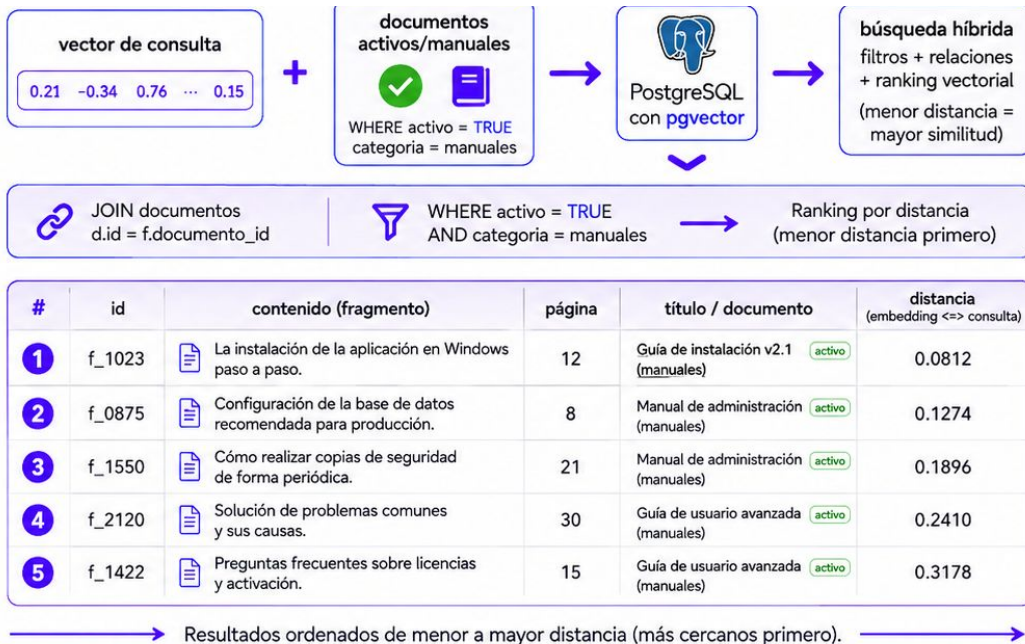
Consulta híbrida

Combinar SQL tradicional y distancia vectorial

```
SELECT
  f.id,
  f.contenido,
  f.pagina,
  d.titulo,
  f.embedding <=> :vector_consulta
AS distancia
FROM fragmentos f
JOIN documentos d
ON d.id = f.documento_id
WHERE d.activo = TRUE
AND d.categoria = 'manuales'
ORDER BY distancia
LIMIT 5;
```

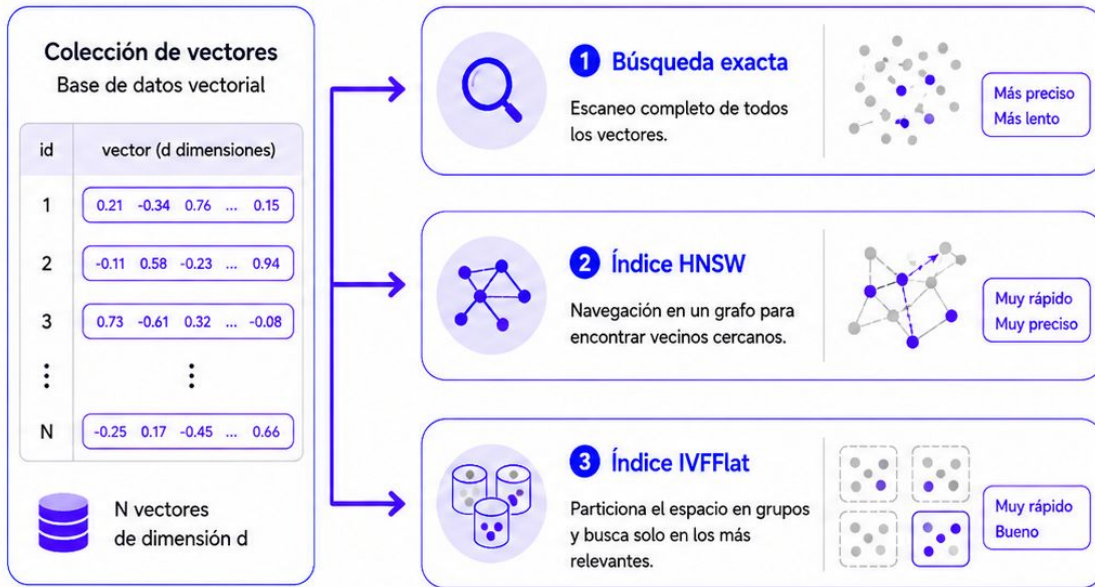
La consulta:

- combina JOIN y filtros relacionales;
- usa pgvector para calcular la distancia;
- ordena por menor distancia;
- devuelve solo los 5 más relevantes.



La búsqueda vectorial no reemplaza filtros, relaciones ni reglas del modelo relacional.

Índices Vectoriales y Rendimiento



Acelerar consultas de similitud.

Búsqueda exacta

Comparar con todos los registros

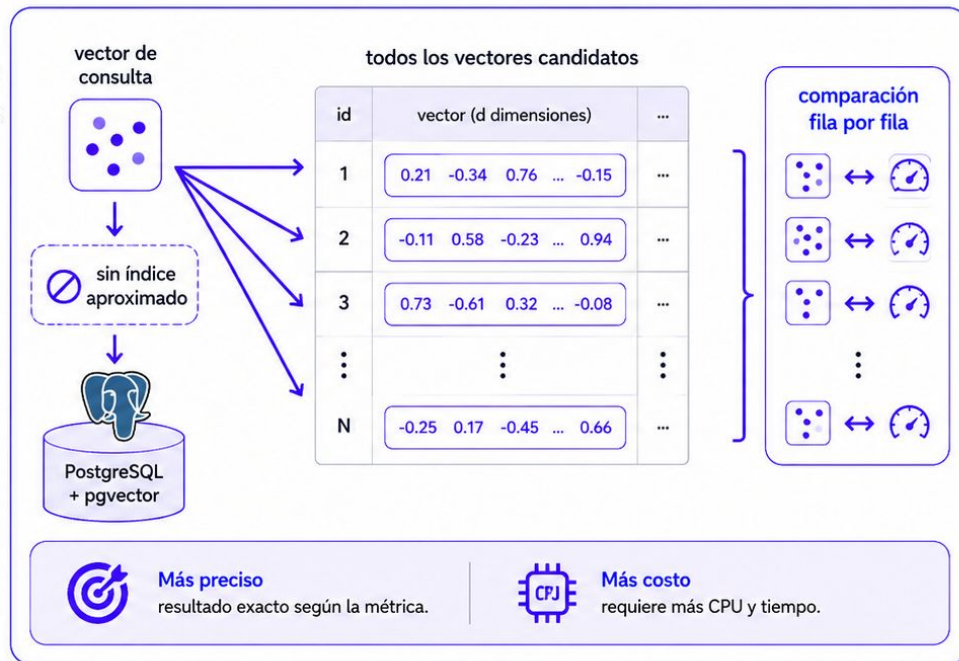
Sin un índice aproximado, PostgreSQL puede calcular la distancia contra todos los vectores candidatos.

Ventajas:

- resultado exacto según la métrica;
- comportamiento predecible;
- no requiere construir índice vectorial.

Limitaciones:

- más CPU;
- mayor latencia;
- menor capacidad concurrente;
- costo creciente con el volumen.



La búsqueda exacta puede ser adecuada para colecciones pequeñas o evaluaciones de referencia.

Índice vectorial

Reducir el espacio de búsqueda

Un índice vectorial crea una estructura auxiliar para evitar comparar todos los registros.

Al igual que otros índices:

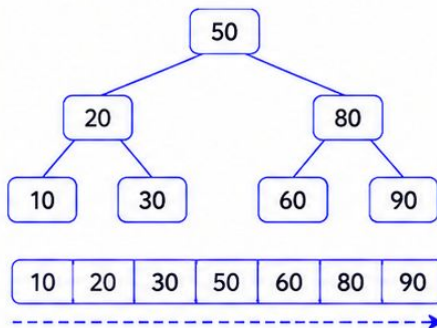
- ocupa espacio;
- requiere construcción;
- debe mantenerse;
- puede acelerar lecturas;
- puede afectar escrituras;
- necesita parámetros adecuados.

Pero, a diferencia de un B-tree, se orienta a proximidad y no a igualdad u orden total.



El índice vectorial evita comparar todos los registros para encontrar los más relevantes.

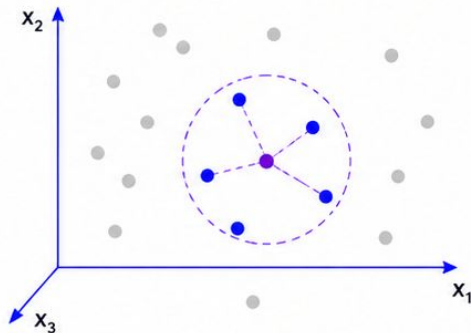
B-tree tradicional



Sirve igualdad y orden total: búsquedas exactas y rangos.

VS.

Índice de proximidad



Sirve proximidad y similitud: búsquedas de vecinos cercanos.



Un índice vectorial es una decisión de diseño físico.

HNSW

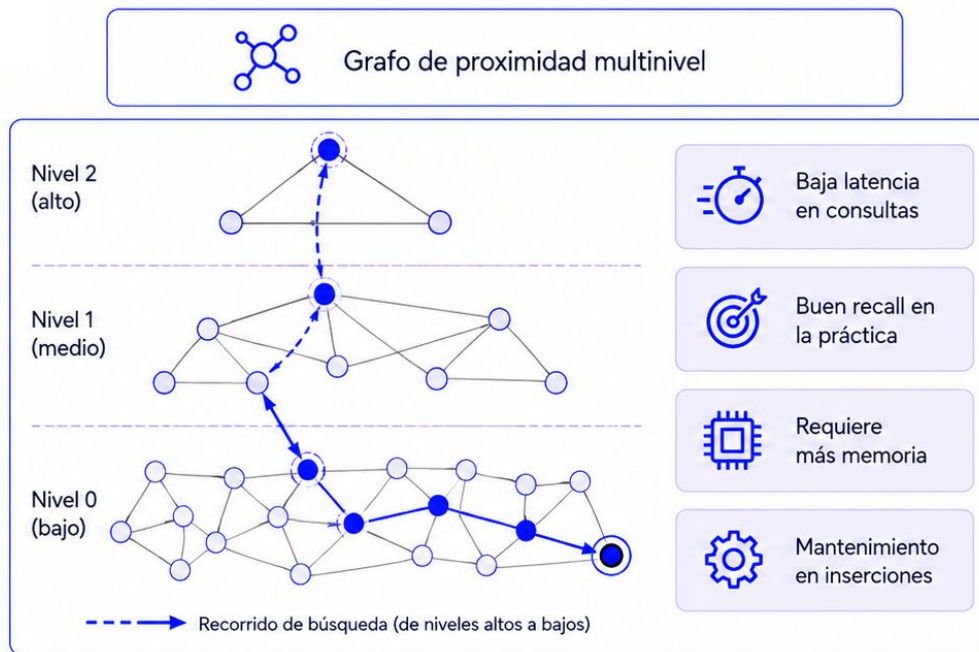
Índice basado en un grafo de proximidad

HNSW organiza los vectores mediante conexiones entre vecinos.

Desde el punto de vista de la base:

- permite consultas rápidas;
- suele ofrecer buen recall;
- requiere más memoria;
- tiene un costo de construcción;
- agrega mantenimiento durante inserciones;
- permite ajustar cuánto explorar al consultar.

No necesitamos implementar el algoritmo, pero sí comprender su impacto operativo.



HNSW prioriza baja latencia a cambio de memoria y mantenimiento del grafo.

CREATE INDEX HNSW

Indexar distancia coseno

```
CREATE INDEX  
idx_fragmentos_embedding_hnsw  
ON fragmentos  
USING hnsw (  
    embedding vector_cosine_ops  
);
```

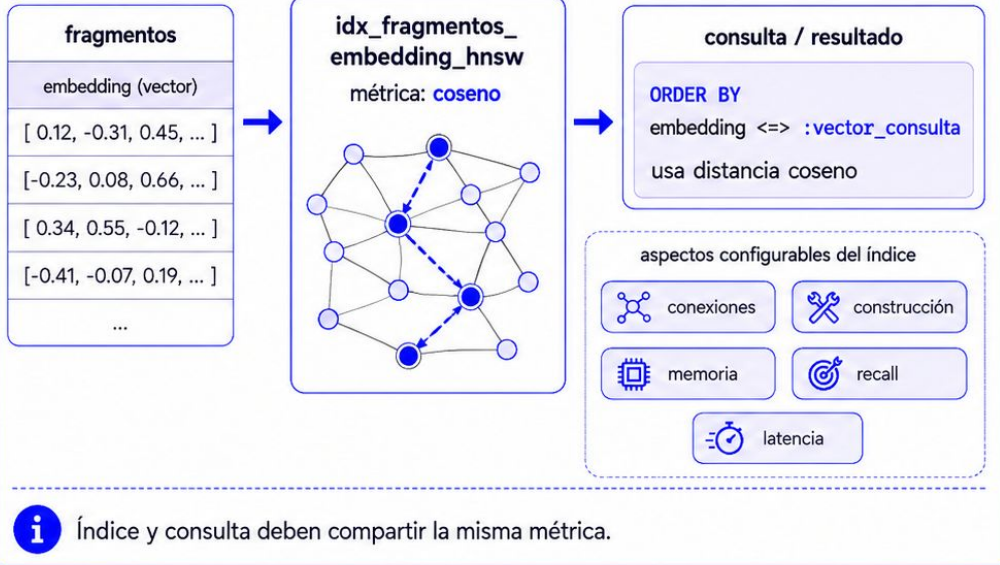
La clase de operador debe coincidir con la distancia usada en las consultas.

El índice puede configurarse con parámetros que afectan:

- conexiones;
- construcción;
- memoria;
- recall;
- latencia.



Índice HNSW con `vector_cosine_ops`



La consulta y el índice deben usar una métrica compatible.

IVFFlat

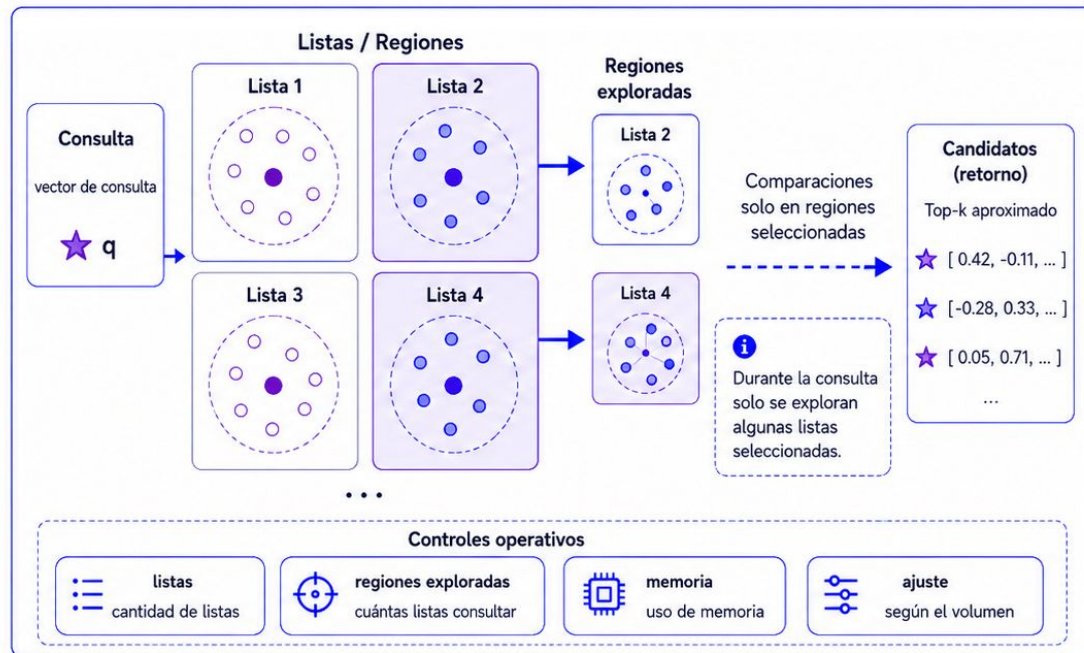
Índice basado en listas y regiones

IVFFlat divide los vectores en grupos.

Durante la consulta se exploran algunas listas en lugar de toda la tabla.

Desde el punto de vista operativo:

- necesita datos para construir una distribución útil;
- requiere definir cantidad de listas;
- permite controlar cuántas regiones explorar;
- ocupa menos memoria que otras alternativas;
- puede necesitar ajuste según el volumen.



IVFFlat reduce comparaciones agrupando vectores en regiones aproximadas.

CREATE INDEX IVFFlat

Configurar cantidad de listas

```
CREATE INDEX  
idx_fragmentos_embedding_ivfflat  
ON fragmentos  
USING ivfflat (  
    embedding vector_cosine_ops  
)  
WITH (lists = 100);
```

La cantidad de listas depende de:

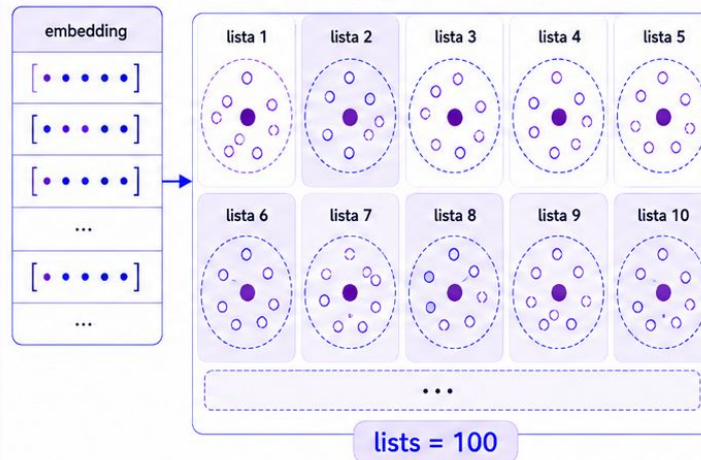
- volumen de registros;
- distribución de datos;
- latencia esperada;
- recall requerido;
- recursos disponibles.



Índice IVFFlat con **lists = 100**

Espacio vectorial particionado en listas
(las listas agrupan vectores candidatos)

fragmentos



Factores prácticos



Más listas

- mayor recall potencial
- más precisión
- mayor costo (memoria y CPU)
- mayor latencia de consulta



Menos listas

- menor costo
- menor latencia
- menor recall potencial



La configuración impacta
latencia, recall y costo.



El número de listas define cómo se particionan los datos antes de la búsqueda.



Un valor de configuración no debe copiarse sin medir el conjunto real de datos.

Comparación

Dos índices para necesidades diferentes



HNSW

- consultas rápidas;
- buen recall;
- mayor uso de memoria;
- mantenimiento más costoso.



IVFFlat

- estructura basada en listas;
- menor consumo relativo;
- requiere configurar distribución;
- puede necesitar reconstrucción o ajuste;
- el resultado depende de cuántas listas se exploran.

Aspecto	HNSW	IVFFlat
 estructura	grafo multinivel	listas / regiones
 velocidad de consulta	muy rápida	rápida
 recall	alto	depende de listas exploradas
 memoria	mayor	menor relativa
 mantenimiento	más costoso	ajuste y posible reconstrucción
 mejor para	baja latencia	menor costo de memoria



Factores clave
para decidir



volumen



escrituras



memoria



latencia



La elección depende del volumen, las escrituras, la memoria y la latencia requerida.

Medir consultas vectoriales

Verificar si el índice se utiliza

Consulta

EXPLAIN ANALYZE

SELECT

id,
contenido

FROM fragmentos

ORDER BY embedding <=> :vector_consulta

LIMIT 5;



Plan de ejecución (ejemplo)

```
Limit (cost=0.42..3.77 rows=5 width=72) (actual time=2.145..2.149 rows=5 loops=1)
-> Index Scan using idx_fragmentos_embedding_hnsw on fragmentos
(cost=0.42..56892.11 rows=100000 width=72) (actual time=2.144..2.146 rows=5 loops=1)
Order By: (embedding <=> '[ ... ]'::vector)
Index Cond: (embedding <=> '[ ... ]'::vector)
```

Resumen

Elemento		Observación
	Recorrido	Utiliza el índice HNSW para ordenar por distancia (Index Scan).
	Filas procesadas	Solo se devuelven 5 filas, pero el índice puede evaluar muchas más.
	Costo estimado	Predicción del optimizador para elegir el plan.
	Tiempo real	Tiempo efectivo de ejecución de cada nodo.
	Ordenamiento	El índice ya entrega los registros ordenados por distancia.
	Impacto del LIMIT	Permite cortar la ejecución al obtener los primeros resultados.



Crear un índice no garantiza que el optimizador lo utilice en todas las consultas.

Ciclo de Vida y Operación



Actualizar vectores sin perder consistencia.

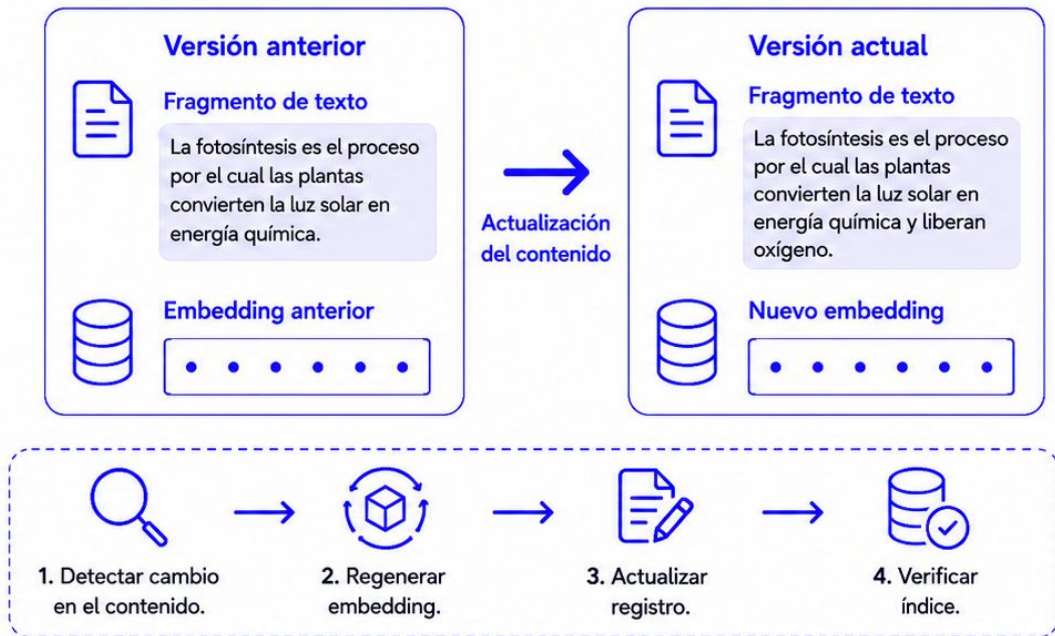
Contenido modificado

El vector también queda desactualizado

Si cambia el contenido de un fragmento, el embedding anterior deja de representar la versión actual.

El proceso puede requerir:

- identificar el contenido modificado;
- generar un nuevo vector;
- actualizar contenido y embedding;
- registrar la nueva versión;
- mantener o eliminar la versión anterior;
- verificar el índice.



Contenido y embedding deben actualizarse como una unidad lógica.

Cambio de modelo de embeddings

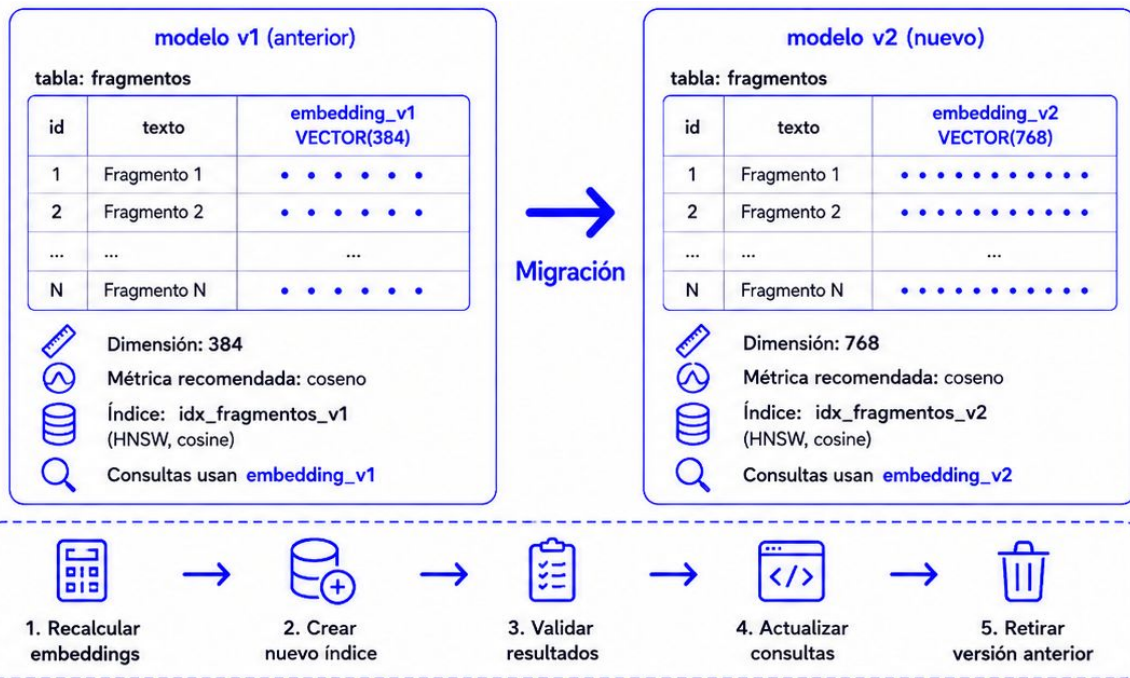
Una migración de datos

Cambiar el modelo puede modificar:

- dimensión;
- distribución de los vectores;
- métrica recomendada;
- resultados de búsqueda;
- tipo de índice necesario.

Una migración puede requerir:

- agregar una nueva columna;
- recalcular todos los embeddings;
- construir un nuevo índice;
- validar resultados;
- cambiar consultas;
- retirar la versión anterior.



Cambiar de modelo no es solo cambiar una API: es migrar datos e índices.

Versionado

Saber con qué representación fue creado cada vector

Conviene registrar:

- modelo de embeddings;
- versión del modelo;
- dimensión;
- fecha de generación;
- versión del documento;
- estrategia de fragmentación;
- estado de indexación.

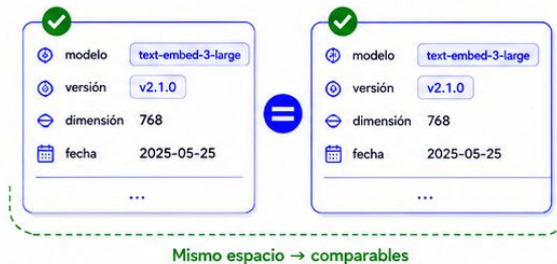
Esto permite:

- auditar resultados;
- reproducir consultas;
- migrar gradualmente;
- detectar registros desactualizados.

Registro de vector (fragmento)		
ID	id_fragmento	12345
....	embedding	
	modelo_embedding	text-embed-3-large
	version_modelo	v2.1.0
	dimension	768
	fecha_generacion	2025-05-25 10:30:45 UTC
	version_documento	v5
	estrategia_fragmentacion	tokens_800_overlap_120
	estado_indexacion	indexado



Comparar solo vectores
del mismo espacio de representación



No comparar vectores
de distintos espacios de representación



Sin versionado, no sabemos si dos vectores pertenecen al mismo espacio de representación.

Eliminar datos vectoriales

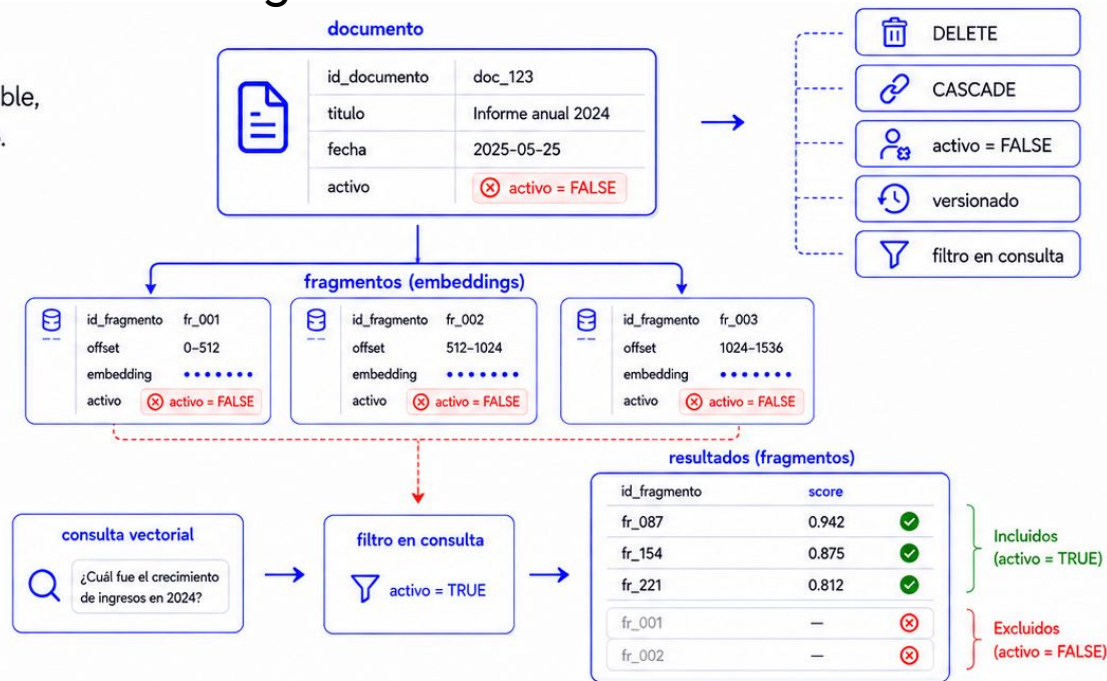
Mantener consistencia con el contenido original

Si un documento se elimina o deja de estar disponible, sus fragmentos no deberían continuar apareciendo.

Alternativas:

- eliminación física con DELETE;
- borrado en cascada;
- baja lógica mediante activo = FALSE;
- conservación histórica con versionado;
- filtros obligatorios en las consultas.

La estrategia depende de auditoría, trazabilidad y cumplimiento.



Un resultado vectorial obsoleto también es un problema de calidad de datos.

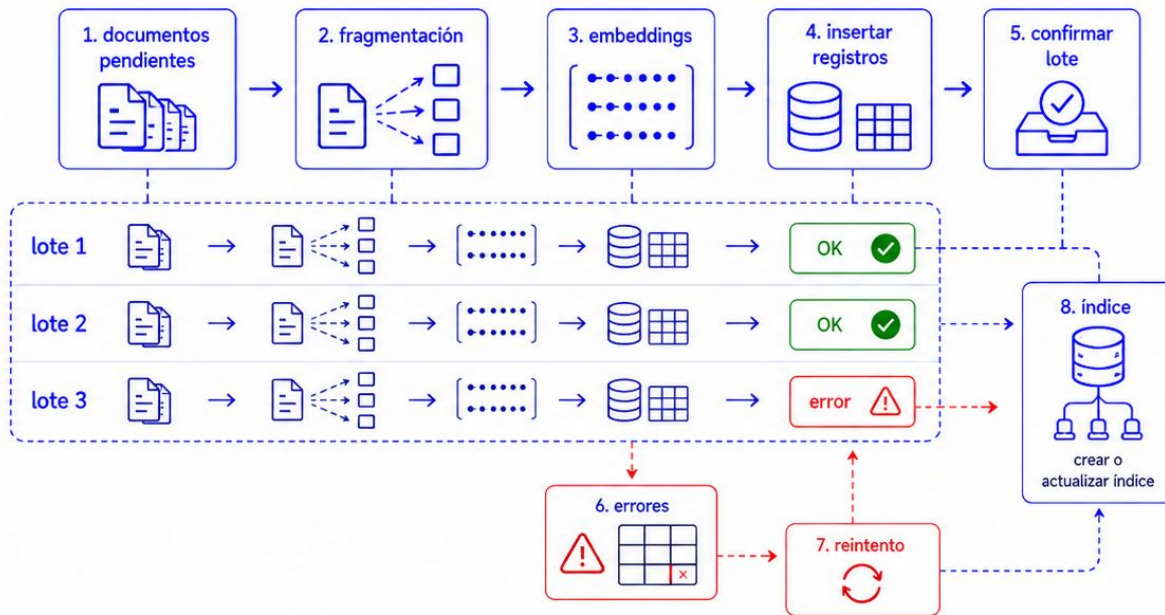
Carga e indexación

Procesar grandes cantidades de documentos

La carga inicial puede realizarse por lotes:

- seleccionar documentos pendientes;
- generar fragmentos;
- calcular embeddings;
- insertar registros;
- confirmar por grupos;
- registrar errores;
- reintentar fallos;
- crear o actualizar índices.

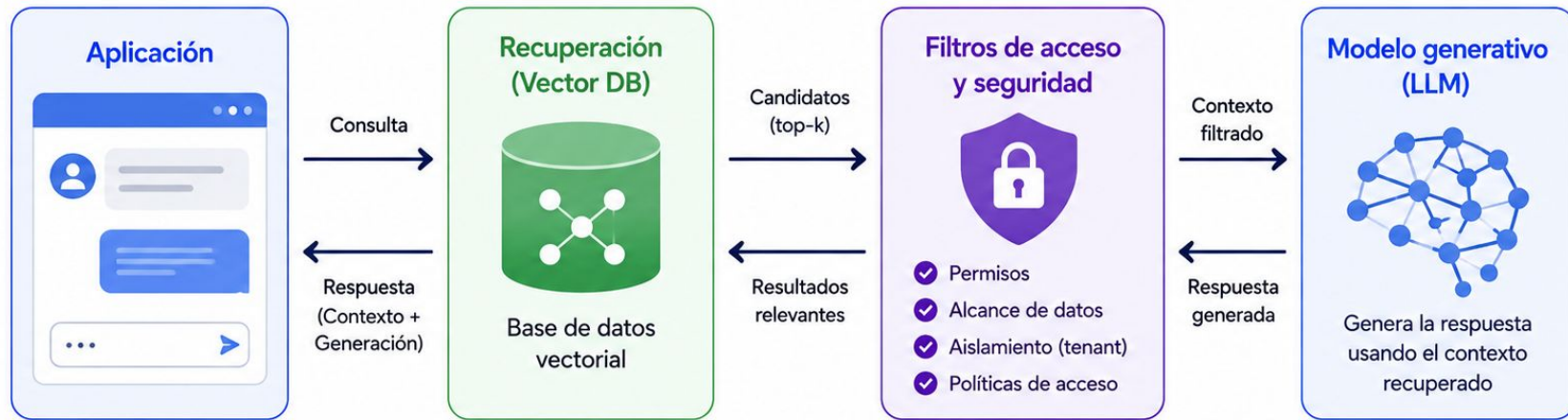
Conviene evitar transacciones excesivamente grandes y mantener trazabilidad del proceso.



La indexación masiva también necesita diseño transaccional y control de errores.

RAG como Arquitectura de Datos

Una aplicación que consume una base vectorial



RAG combina recuperación de información relevante con generación de respuestas.

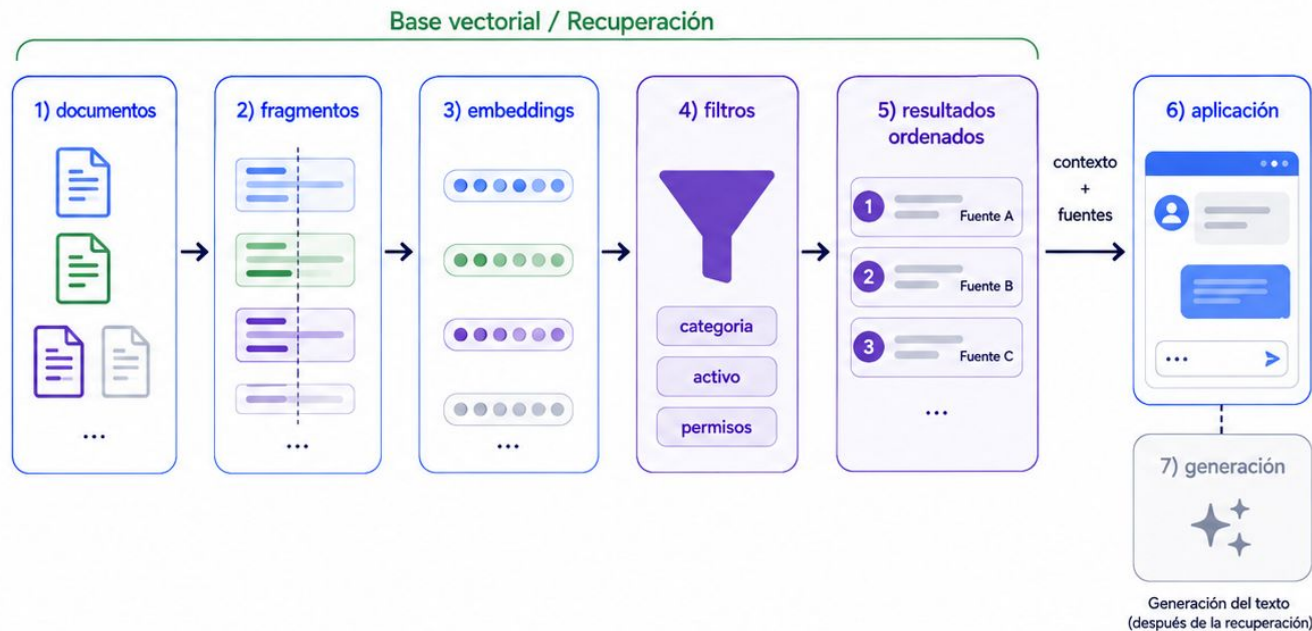
RAG

Un caso de consumo de datos vectoriales

Desde el punto de vista de la base, un sistema RAG necesita:

- almacenar documentos;
- dividirlos en fragmentos;
- guardar embeddings;
- aplicar filtros;
- recuperar resultados ordenados;
- devolver contenido y fuentes;
- respetar permisos;
- actualizar información;
- mantener trazabilidad.

La generación del texto ocurre después de la recuperación.



La responsabilidad de la base es recuperar contexto correcto, vigente y autorizado.

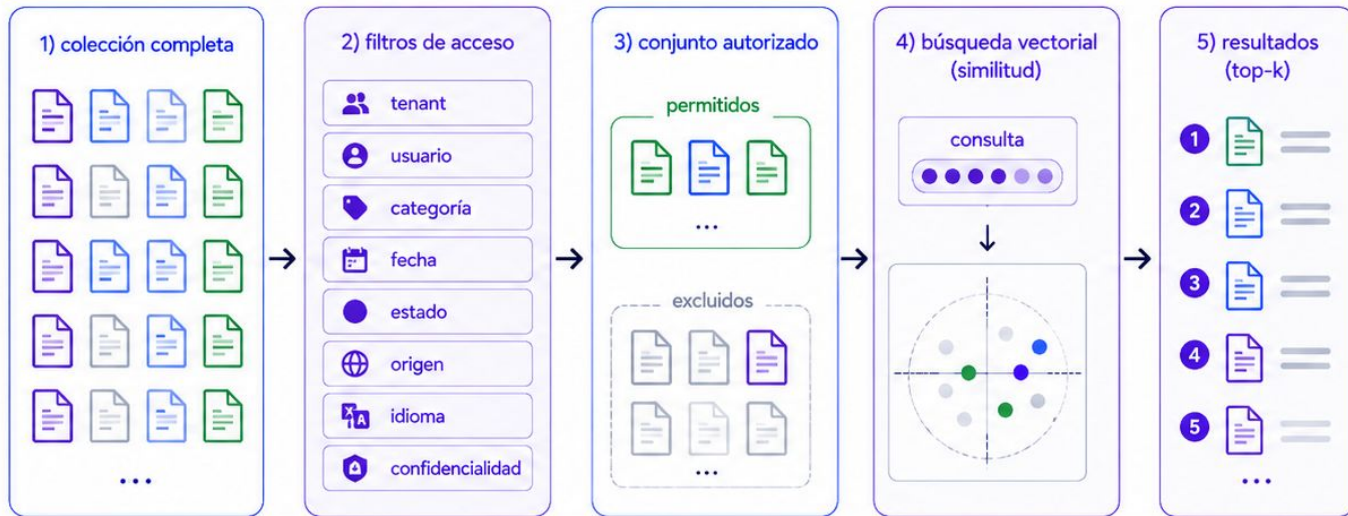
Filtrar antes de recuperar

La similitud no reemplaza las reglas de acceso

Una consulta puede limitar los candidatos por:

- tenant;
- usuario;
- categoría;
- fecha;
- estado;
- origen;
- idioma;
- nivel de confidencialidad.

Luego, dentro del conjunto autorizado, se aplica el ordenamiento por similitud.



Primero definimos qué datos puede ver el usuario.
Después buscamos cuáles son los más parecidos.

Trazabilidad

Volver desde el resultado hacia la fuente

Cada fragmento recuperado debería permitir identificar:

- documento de origen;
- página o sección;
- versión;
- fecha;
- autor;
- categoría;
- identificador interno.

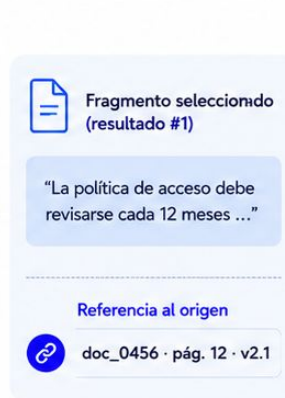
Esto permite:

- citar fuentes;
- verificar respuestas;
- revisar errores;
- actualizar documentos;
- auditar accesos.

1) resultado recuperado



2) fragmento / referencia



3) documento de origen (metadatos)



4) usos habilitados por la trazabilidad



Una búsqueda útil debe poder explicar de dónde salió cada fragmento.

Problemas frecuentes

Cuando la falla está en la capa de datos

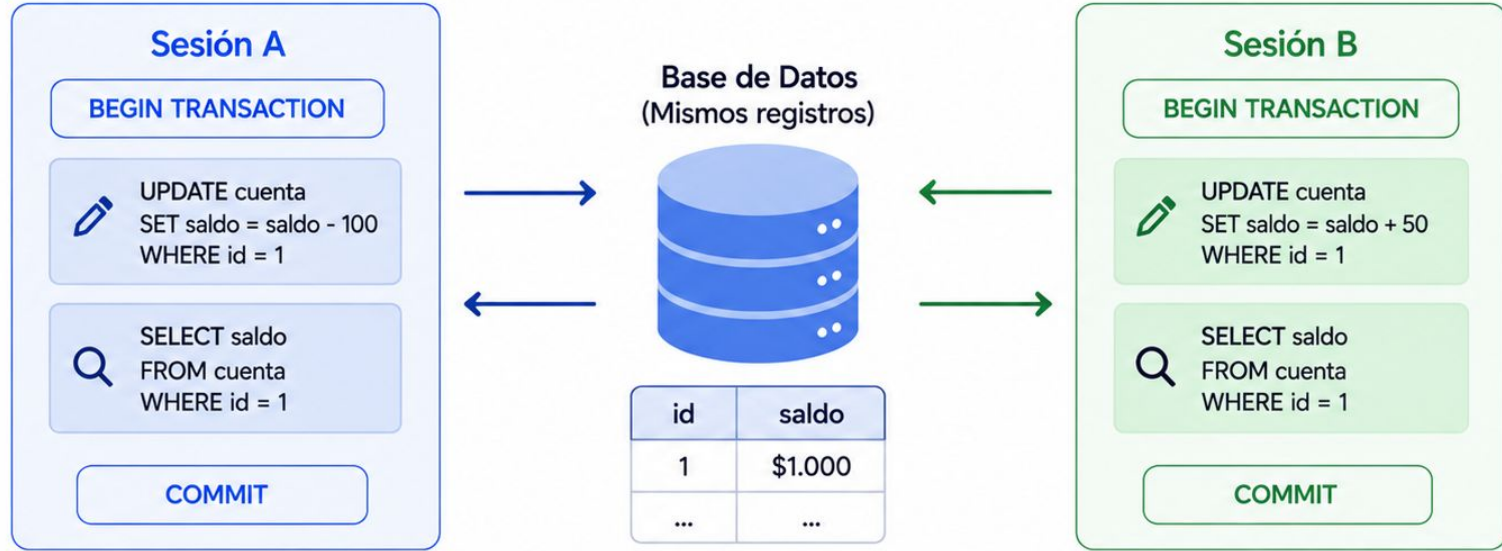
Un sistema puede devolver malos resultados si:

- el documento no fue cargado;
- la versión está desactualizada;
- los fragmentos son demasiado grandes;
- faltan metadatos;
- el filtro excluye contenido válido;
- el embedding pertenece a otro modelo;
- el índice no fue actualizado;
- existen duplicados;
- el dato fue eliminado pero sigue activo.



Antes de cambiar el modelo, conviene revisar la calidad y operación de la base vectorial.

Transacciones y Concurrency



Las transacciones bien definidas y el control de concurrencia evitan inconsistencias.

Transacciones y ACID

Repaso breve

Una transacción agrupa operaciones que deben tratarse como una unidad.

ACID:

- **Atomicidad:** todo o nada.
- **Consistencia:** las reglas se respetan.
- **Aislamiento:** operaciones concurrentes no producen interferencias incorrectas.
- **Durabilidad:** lo confirmado persiste.

En esta sección vamos a profundizar especialmente el **aislamiento**.

A

Atomicidad



Todo o nada.

C

Consistencia



Las reglas se respetan.

I

Aislamiento



Operaciones concurrentes no producen interferencias incorrectas.

D

Durabilidad



Lo confirmado persiste.

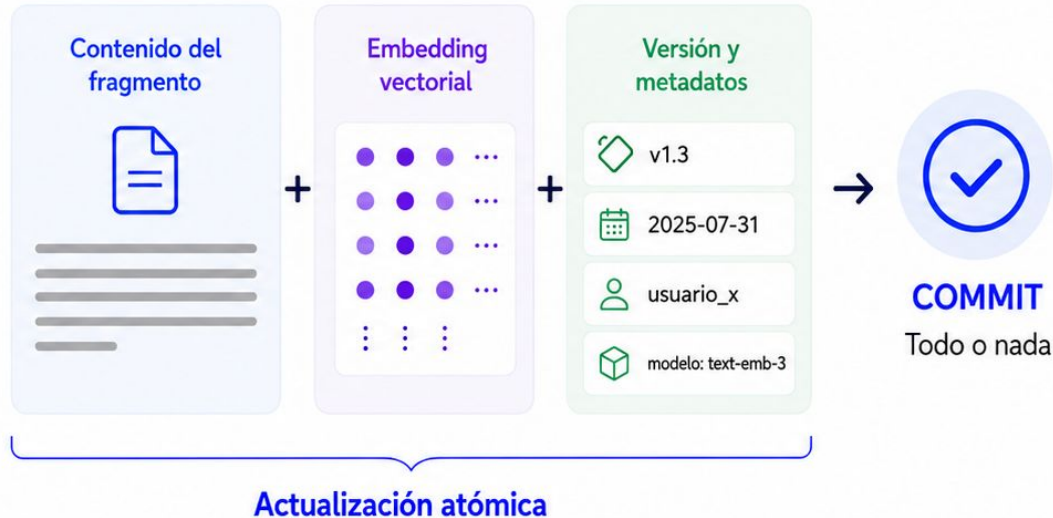


El aislamiento define qué puede observar una transacción mientras otras todavía están trabajando.

Actualización consistente

Contenido, vector y versión en una misma transacción

```
BEGIN;  
UPDATE fragmentos  
SET  
  contenido = :nuevo_contenido,  
  embedding = :nuevo_embedding,  
  modelo_embedding = :modelo,  
  fecha_indexacion = CURRENT_TIMESTAMP  
WHERE id = :fragmento_id;  
COMMIT;
```



Si una parte falla, podemos ejecutar **ROLLBACK** y evitar que contenido y vector queden desincronizados.



Las transacciones también protegen el ciclo de vida de los embeddings.

Actualización perdida

Cuando una escritura sobrescribe a otra

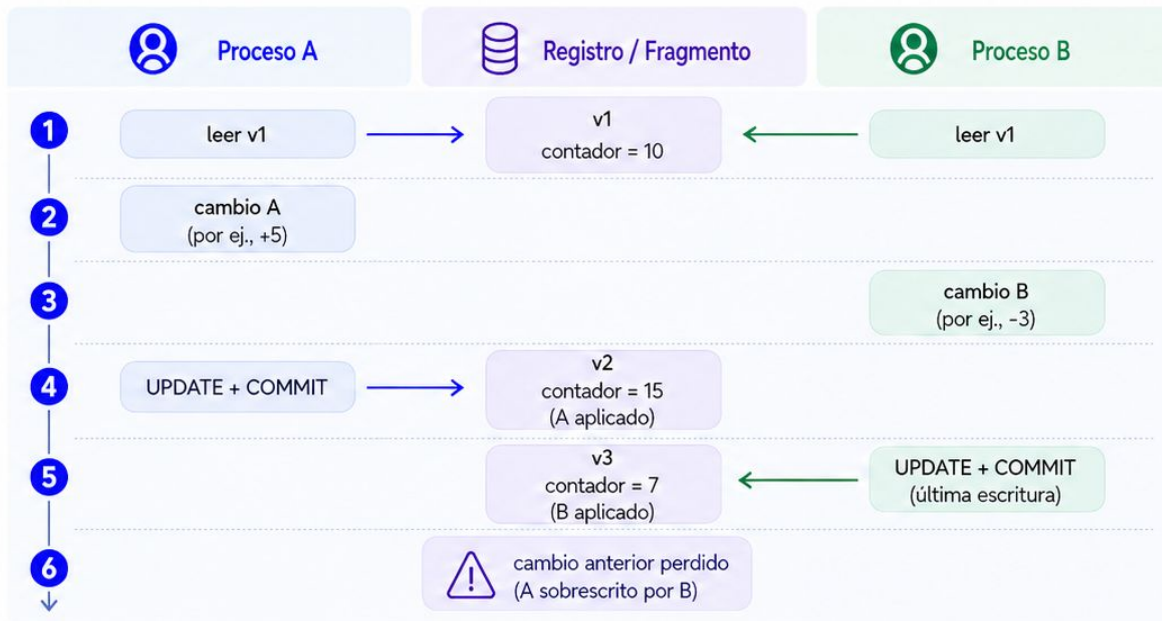
Dos procesos pueden:

- leer la misma versión;
- calcular cambios diferentes;
- actualizar el mismo registro;
- confirmar en momentos distintos.

La última escritura puede eliminar el cambio anterior.

Esto puede afectar:

- estados;
- contadores;
- versiones;
- metadatos;
- procesos de reindexación.



La concurrencia puede producir un resultado válido en formato, pero incorrecto en contenido.

Lectura no repetible

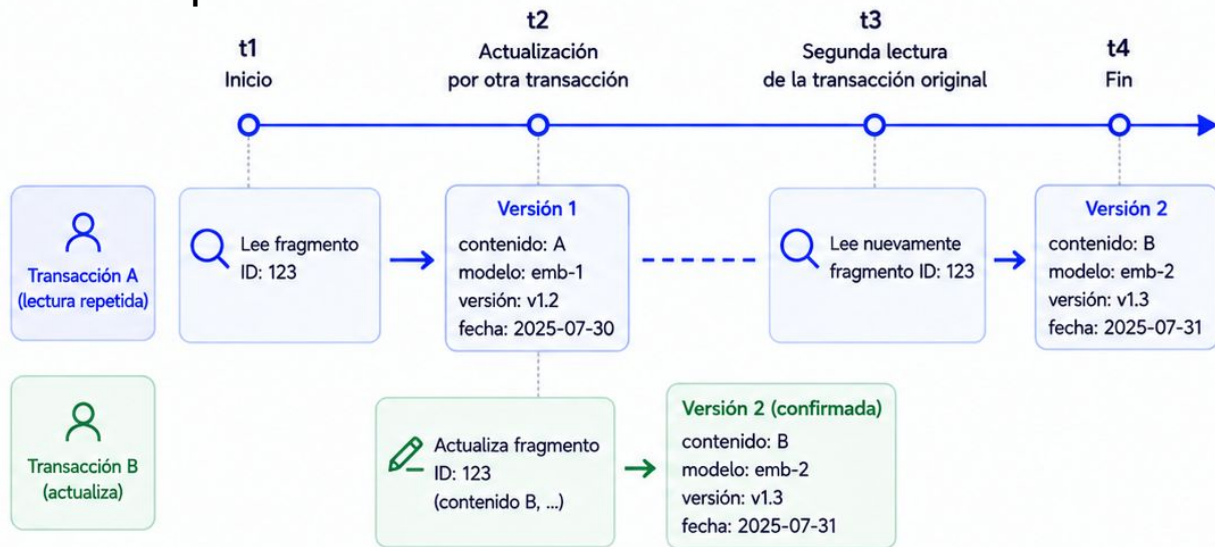
El mismo registro cambia durante la operación

Una transacción consulta un fragmento y obtiene una versión.

Mientras continúa trabajando, otra transacción actualiza y confirma ese registro.

Si la primera consulta nuevamente, puede observar contenido o metadatos diferentes.

Esto puede afectar procesos que necesitan consistencia durante una misma ejecución.



La misma transacción puede observar dos estados confirmados diferentes.

Lecturas fantasma

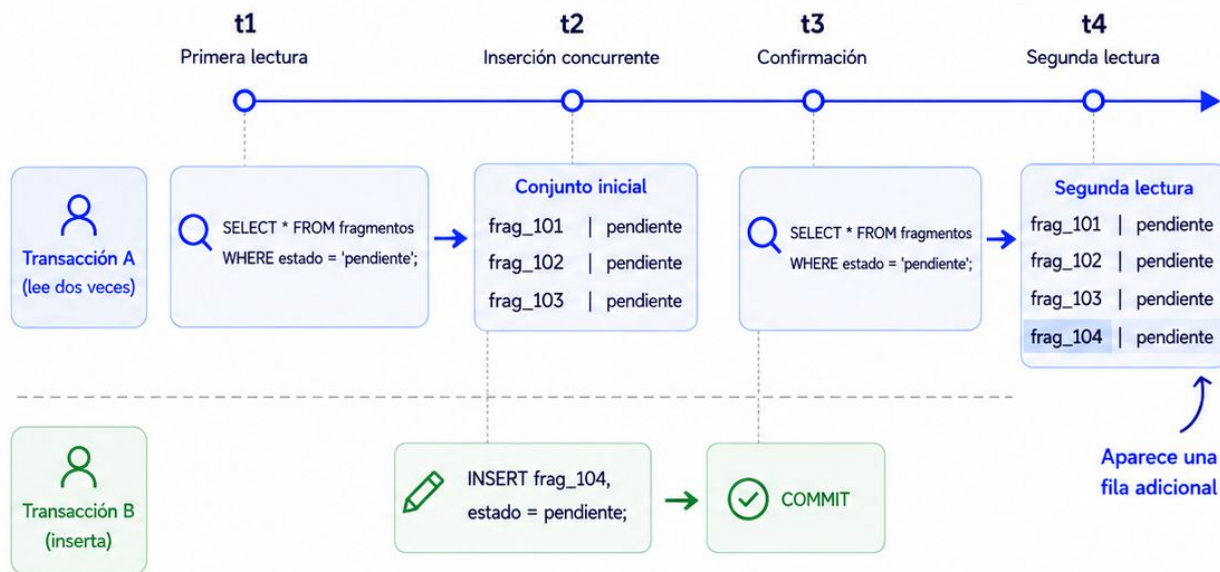
El conjunto de filas cambia

Una transacción consulta todos los fragmentos pendientes de indexación.

Otra transacción inserta nuevos registros que cumplen esa condición.

Al repetir la consulta, aparecen filas adicionales.

El cambio no ocurre dentro de una fila, sino dentro del conjunto seleccionado.



Las consultas por condición también pueden cambiar durante una transacción.

Bloqueos

Coordinar operaciones sobre recursos compartidos

Los bloqueos pueden aplicarse sobre:

- filas;
- tablas;
- páginas;
- estructuras internas.

Permiten evitar ciertas interferencias, pero introducen:

- esperas;
- menor concurrencia;
- riesgo de deadlocks;
- mayor latencia.



Efectos de los bloqueos



Espera

Las transacciones deben esperar al recurso.



Menor concurrencia

Menos operaciones pueden ejecutarse a la vez.



Riesgo de deadlocks

Transacciones pueden bloquearse entre sí.



Mayor latencia

Las operaciones tardan más en completarse.

Ejemplo de deadlock



Bloquear protege consistencia, pero reduce el trabajo que puede ejecutarse al mismo tiempo.

Deadlocks

Transacciones esperando entre sí

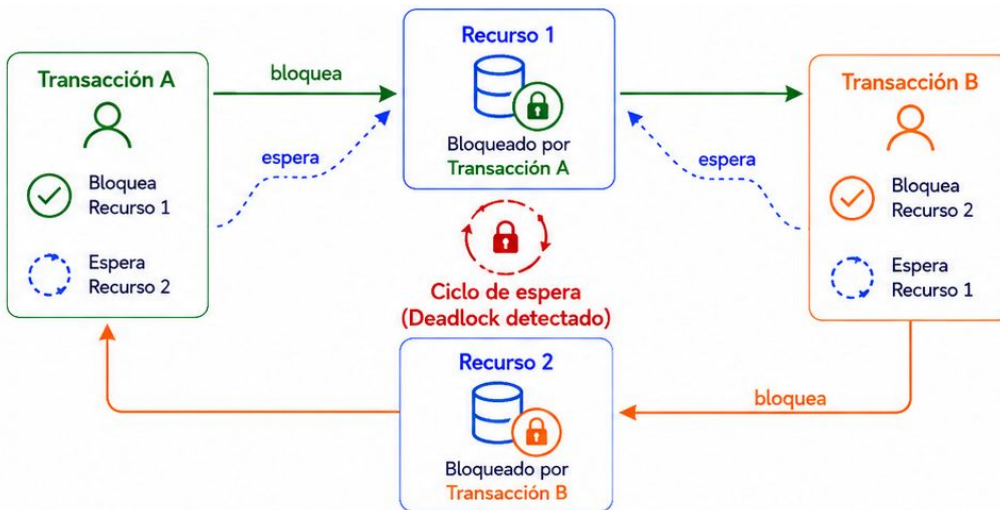
Un deadlock puede ocurrir cuando:

- la transacción A bloquea el recurso 1;
- la transacción B bloquea el recurso 2;
- A espera el recurso 2;
- B espera el recurso 1.

PostgreSQL detecta el ciclo y cancela una operación.

La aplicación debe estar preparada para:

- recibir el error;
- deshacer;
- reintentar;
- registrar el conflicto.



Respuesta del sistema (PostgreSQL)



Detecta el ciclo de esperas (deadlock) y **cancela** una de las transacciones.



Error (deadlock)

Flujo en la aplicación



Deshacer (rollback)



Reintentar (retry)



Registrar el conflicto



Un deadlock no se resuelve esperando indefinidamente: una transacción debe retroceder.

MVCC

Versiones para permitir lecturas concurrentes

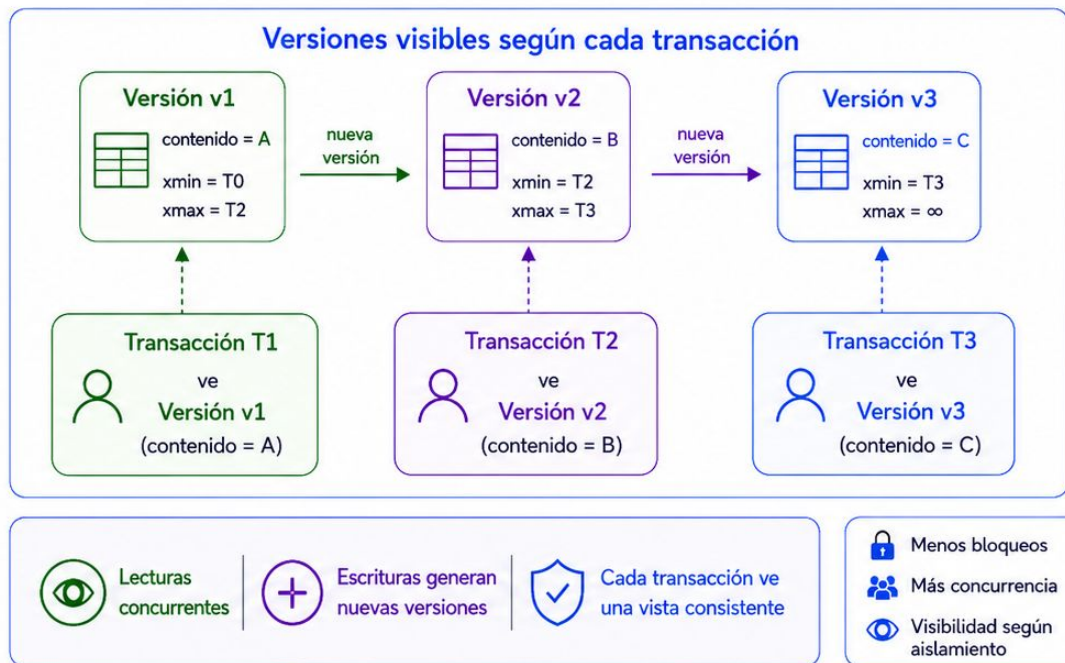
PostgreSQL utiliza Multiversion Concurrency Control.

En lugar de reemplazar inmediatamente una fila para todos los usuarios, mantiene versiones visibles según cada transacción.

Esto permite:

- reducir bloqueos entre lecturas y escrituras;
- mantener vistas consistentes;
- mejorar concurrencia;
- controlar qué cambios pueden observarse.

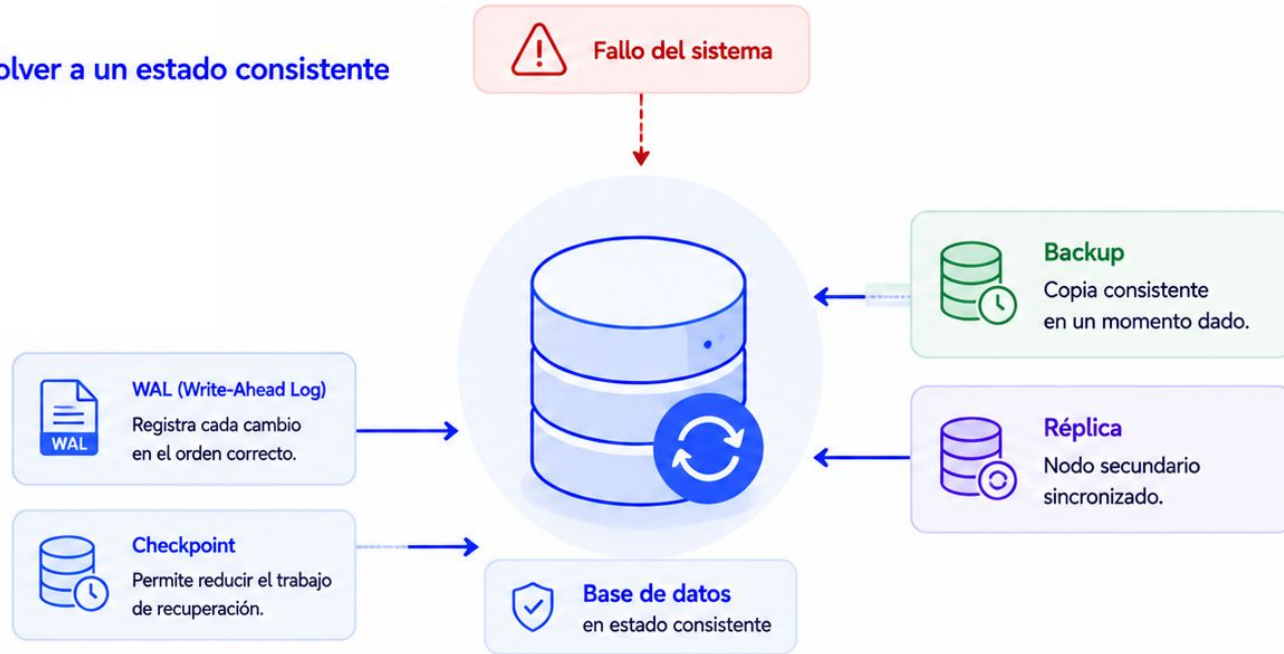
Los niveles de aislamiento definen cuánto se protege cada transacción.



MVCC permite compartir la base manteniendo distintas vistas consistentes de los datos.

Recuperación ante fallos

Volver a un estado consistente



La recuperabilidad permite restaurar la base y continuar operando.
La tolerancia a fallos asegura disponibilidad ante eventos inesperados.

Qué puede fallar

Diferentes fallos requieren diferentes respuestas

Un sistema puede enfrentar:

- error dentro de una transacción;
- caída de una aplicación;
- reinicio del servidor;
- pérdida de conexión;
- corrupción de archivos;
- falla de disco;
- error humano;
- eliminación accidental;
- caída completa del sitio.

No todos estos problemas se resuelven con el mismo mecanismo.



Antes de diseñar recuperación, debemos identificar de qué falla queremos recuperarnos.

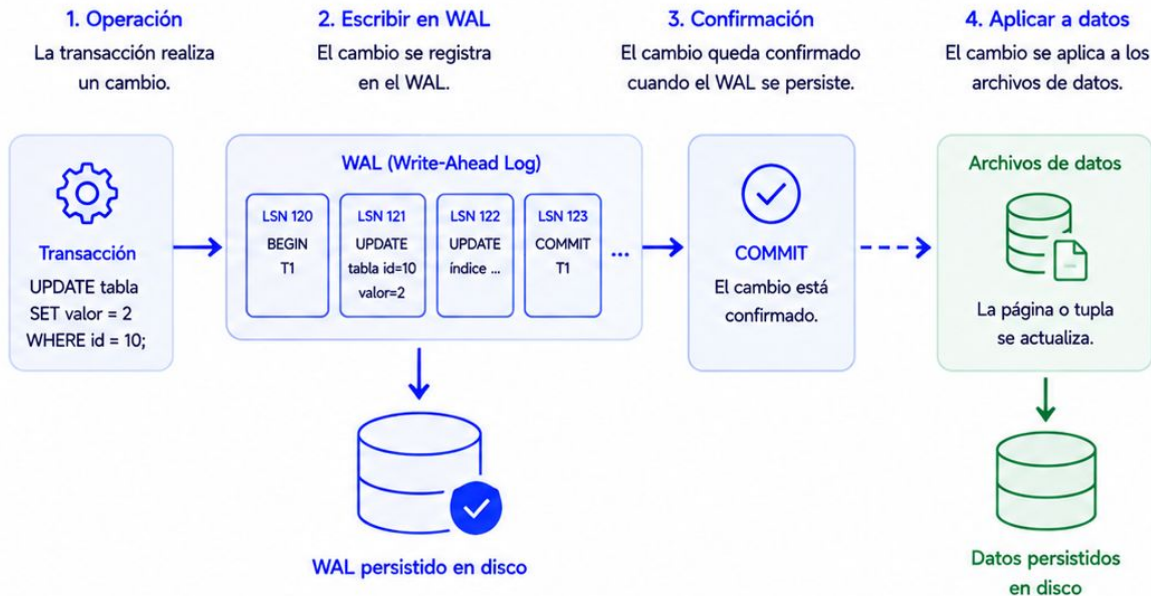
Write-Ahead Logging

Registrar antes de consolidar

PostgreSQL registra cambios en el WAL antes de considerarlos persistentes.

Después de una caída, puede:

- rehacer operaciones confirmadas;
- ignorar transacciones incompletas;
- reconstruir un estado consistente;
- enviar cambios a réplicas;
- soportar recuperación a un punto en el tiempo.



La durabilidad depende de registrar correctamente los cambios antes de aplicarlos definitivamente.

Checkpoints

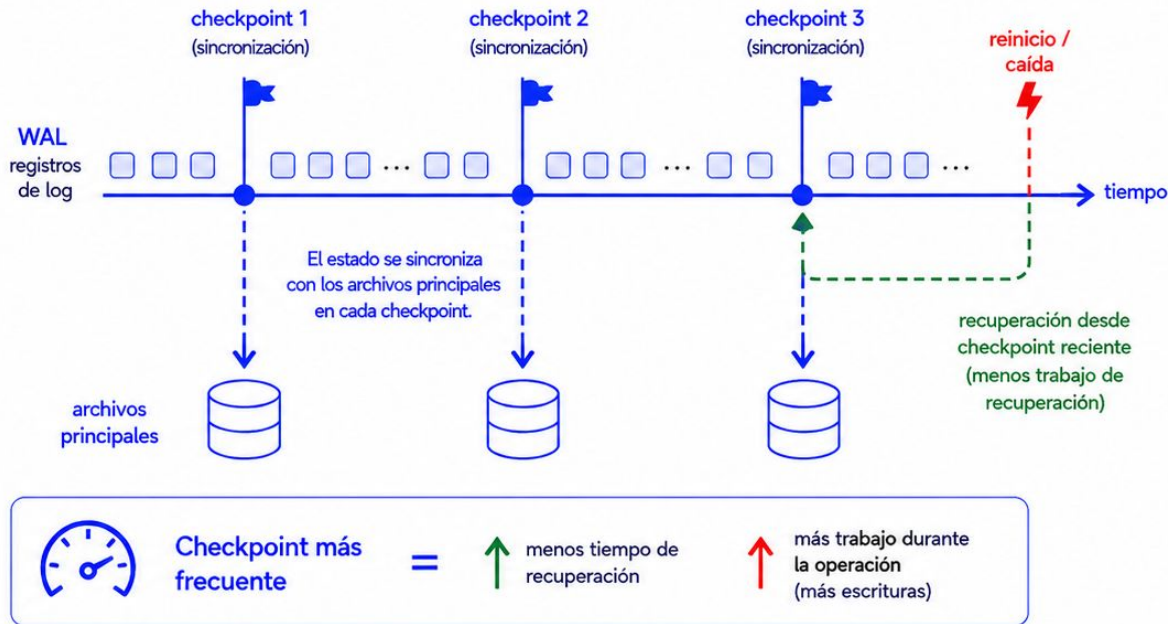
Reducir el trabajo posterior a una caída

Un checkpoint marca un punto en el que parte del estado fue sincronizado con los archivos principales.

Después de un reinicio, PostgreSQL puede comenzar la recuperación desde un punto reciente.

La frecuencia de checkpoints influye en:

- tiempo de recuperación;
- cantidad de escritura;
- uso del WAL;
- rendimiento del sistema.



Un checkpoint reduce recuperación futura, pero también genera trabajo durante la operación.

Backup, WAL y réplica

Mecanismos complementarios



Backup

- conserva una copia restaurable.



WAL

- permite recuperar cambios y reproducir operaciones.



Réplica

- mantiene otra instancia con datos similares.

Ninguno reemplaza completamente a los demás:

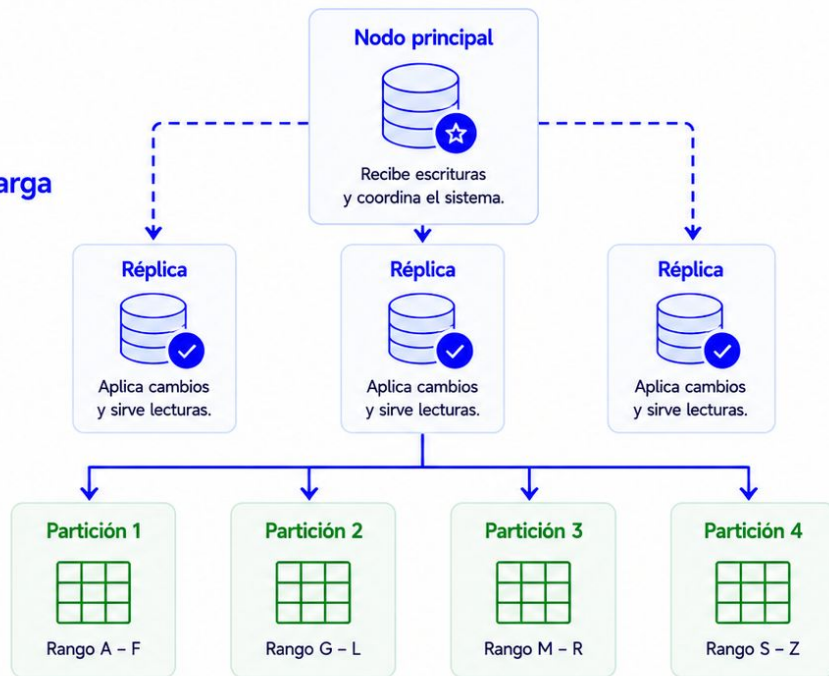
- una réplica puede copiar un borrado accidental;
- un backup puede estar desactualizado;
- el WAL no reemplaza una política de retención.



Alta disponibilidad y recuperación histórica son problemas diferentes.

Distribución y Escalabilidad

Distribuir copias, datos y carga



Distribuir copias mejora disponibilidad.

Distribuir datos y carga mejora rendimiento y permite crecer horizontalmente.

Escalabilidad

Repaso breve

Escalabilidad vertical:

- más CPU;
- más memoria;
- discos más rápidos;
- un servidor más potente.

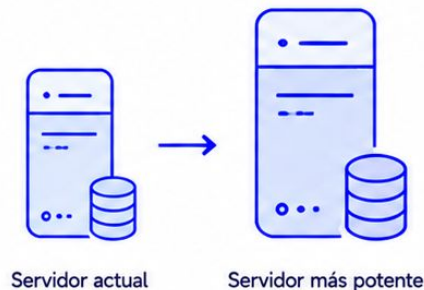
Escalabilidad horizontal:

- más nodos;
- distribución de lecturas;
- distribución de datos;
- coordinación entre servidores.

La primera es más simple.

La segunda permite crecer más, pero introduce complejidad distribuida.

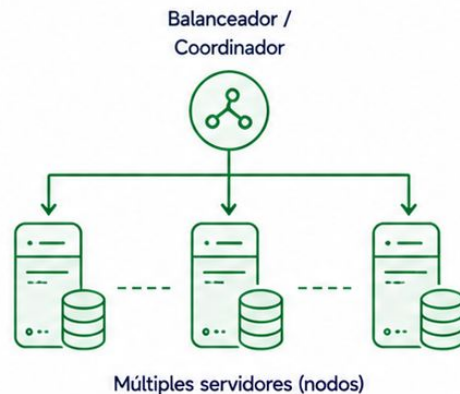
Escalabilidad vertical



Mejora en el mismo servidor aumentando sus recursos.

VS

Escalabilidad horizontal



Distribuye cargas y datos entre varios servidores.



Escalar horizontalmente exige decidir cómo se distribuyen datos, consultas y fallos.

Replicación

Copias de los mismos datos

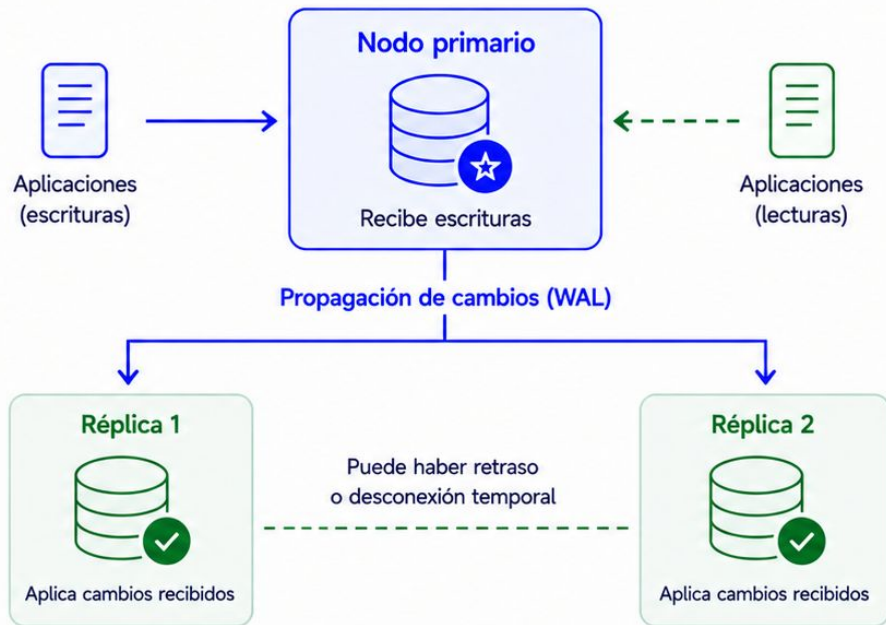
La replicación puede utilizarse para:

- alta disponibilidad;
- distribuir lecturas;
- recuperación ante fallos;
- mantenimiento;
- proximidad geográfica.

Un nodo principal puede recibir escrituras y enviar cambios a réplicas.

Las réplicas pueden estar:

- actualizadas;
- retrasadas;
- temporalmente desconectadas.



Replicar mejora disponibilidad, pero introduce sincronización y retraso.

Síncrona y asíncrona

Cuándo confirmar una escritura

Síncrona

- espera confirmación de otra instancia;
- reduce riesgo de pérdida;
- aumenta latencia;
- depende de la disponibilidad de la réplica.



La escritura se confirma solo después de recibir la confirmación de la réplica.

Asíncrona

- confirma antes de completar la copia;
- reduce latencia;
- puede producir retraso;
- puede perder cambios recientes ante una falla.



La escritura se confirma antes de que la réplica aplique el cambio.



La confirmación más segura suele requerir más tiempo y coordinación.

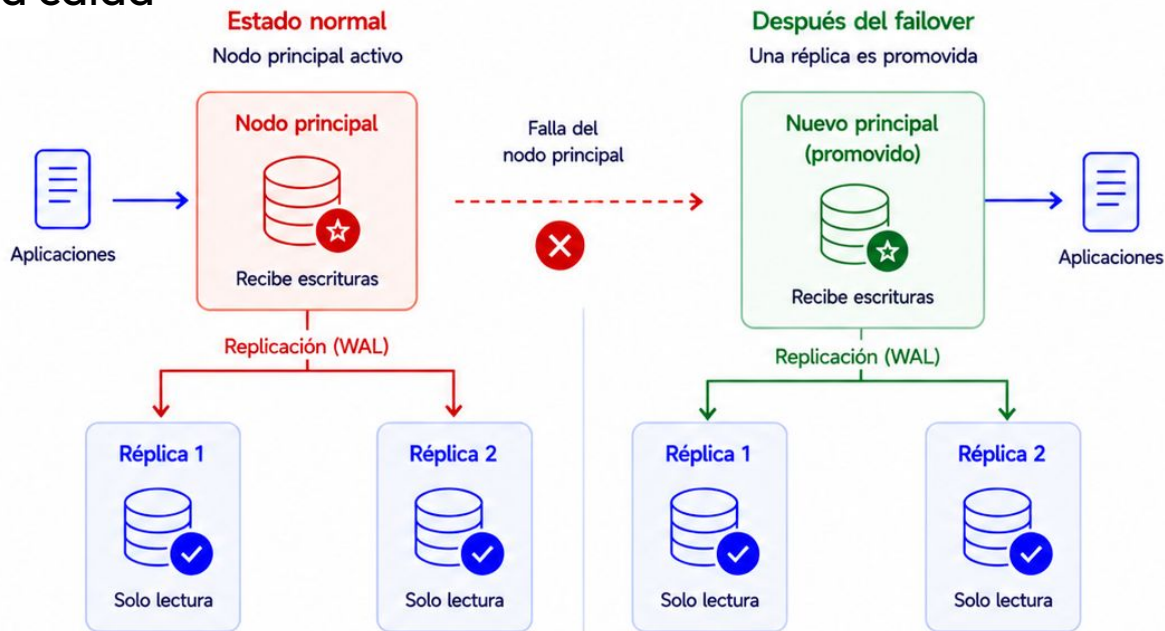
Failover

Promover una réplica ante una caída

Si el nodo principal falla, una réplica puede convertirse en el nuevo principal.

El proceso debe resolver:

- detección de la falla;
- elección de la réplica;
- verificación del retraso;
- promoción;
- reconexión de aplicaciones;
- prevención de dos líderes;
- reintegración del nodo anterior.



Failover no es solamente encender otro servidor: es coordinar un cambio de autoridad.

Particionamiento

Dividir los datos

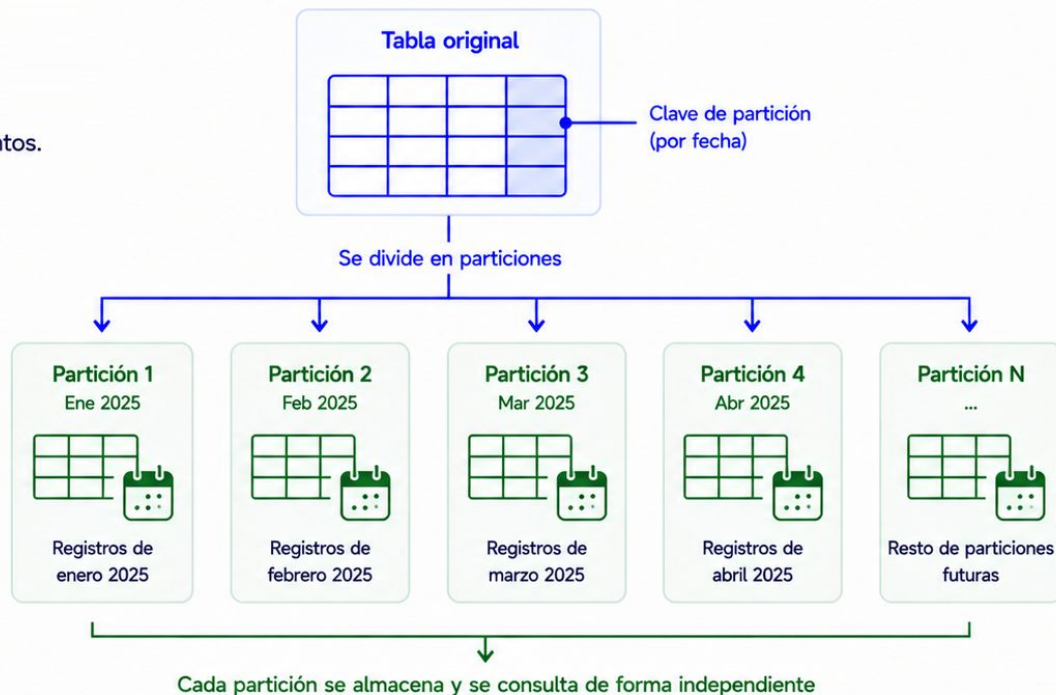
El particionamiento separa registros en subconjuntos.

Puede realizarse por:

- rango;
- fecha;
- hash;
- lista;
- región;
- tenant.

Una buena clave de partición busca:

- distribuir carga;
- evitar hotspots;
- facilitar consultas;
- limitar movimiento de datos;
- permitir crecimiento.



Replicar copia datos.
Particionar divide datos.

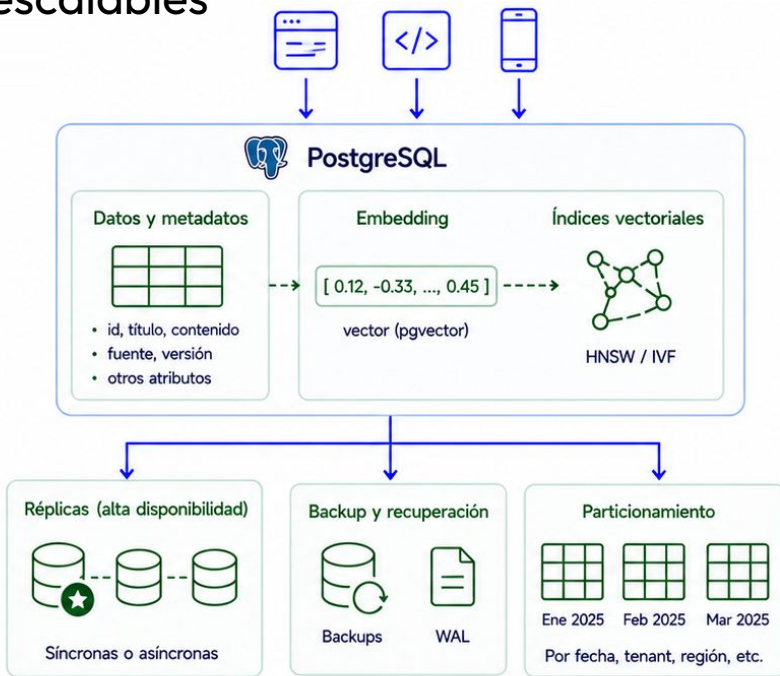


Para ir cerrando..

Buenas prácticas

Criterios para bases vectoriales y sistemas escalables

- Modelar documentos y fragmentos antes de crear vectores.
- Vincular cada embedding con contenido, fuente y versión.
- Registrar el modelo utilizado.
- Usar columnas relacionales para filtros importantes.
- Crear índices solo después de medir.
- Mantener consulta e índice con métricas compatibles.
- Actualizar contenido y embedding de forma consistente.
- Separar backup, recuperación y alta disponibilidad.
- Elegir replicación o particionamiento según el problema.
- Mantener la arquitectura más simple que resuelva la carga.

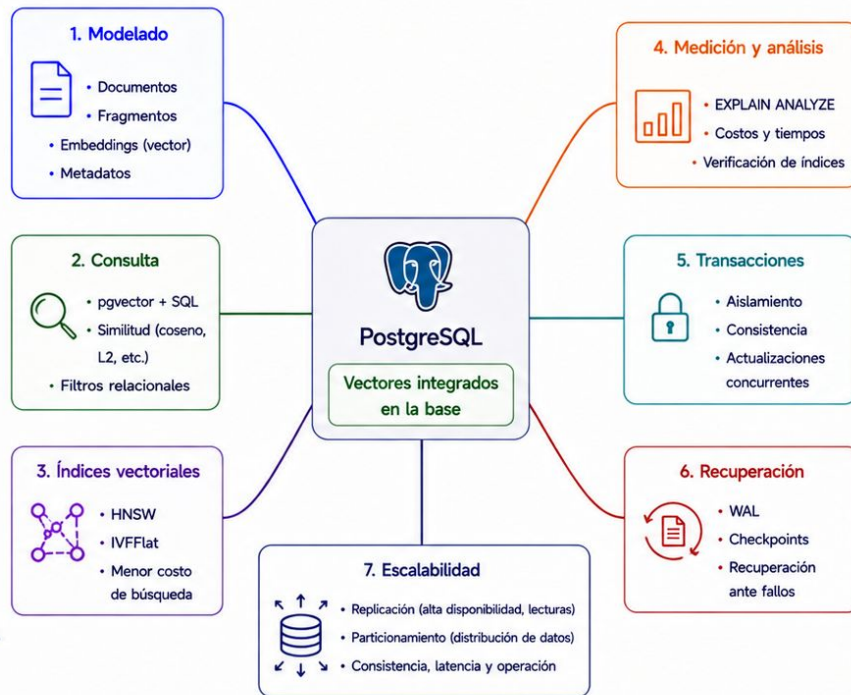


Una base vectorial sigue necesitando buen modelado, integridad, operación y medición.

Resumen de la clase

Ideas principales

- Los vectores pueden incorporarse como un tipo de dato dentro de PostgreSQL.
- El modelo debe representar documentos, fragmentos, embeddings y metadatos.
- pgvector permite consultar similitud mediante SQL.
- Los filtros relacionales siguen siendo necesarios.
- HNSW e IVFFlat reducen el costo de búsqueda.
- EXPLAIN ANALYZE permite medir consultas vectoriales.
- Cambiar el modelo puede exigir migrar datos e índices.
- Las transacciones protegen actualizaciones concurrentes.
- WAL y checkpoints permiten recuperar la base.
- Replicación y particionamiento resuelven problemas diferentes.
- Escalar exige controlar consistencia, latencia y operación.



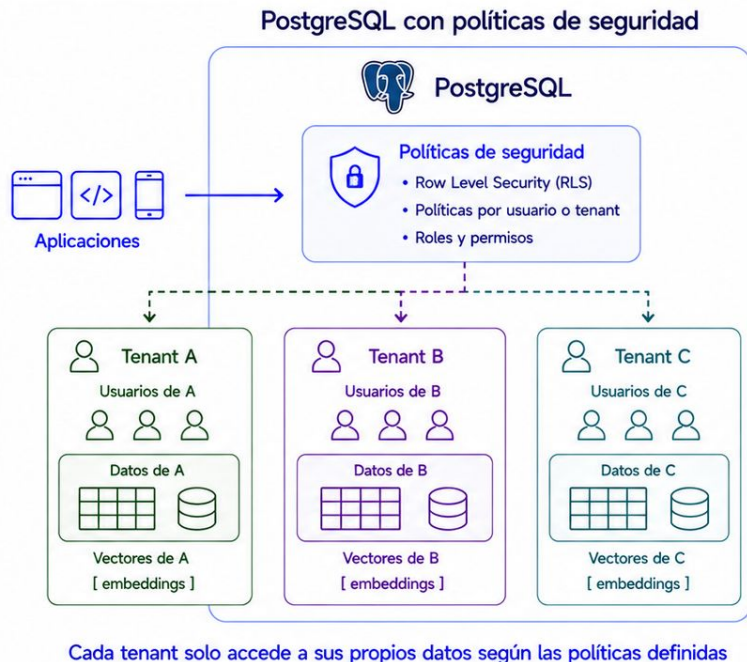
La búsqueda vectorial no reemplaza los fundamentos de bases de datos: los vuelve todavía más importantes.

Próxima clase

Seguridad aplicada para Bases de Datos

En la próxima clase vamos a trabajar sobre:

- arquitecturas multi-tenant;
- aislamiento de datos;
- roles y permisos;
- Row Level Security;
- políticas por usuario o tenant;
- acceso seguro desde aplicaciones;
- filtros de seguridad en consultas vectoriales;
- aislamiento de documentos en sistemas RAG;
- riesgos de filtración entre usuarios;
- tecnologías especializadas y buenas prácticas.



Hoy vimos cómo almacenar, consultar y escalar.

La próxima clase veremos cómo garantizar que cada usuario acceda solamente a sus datos.



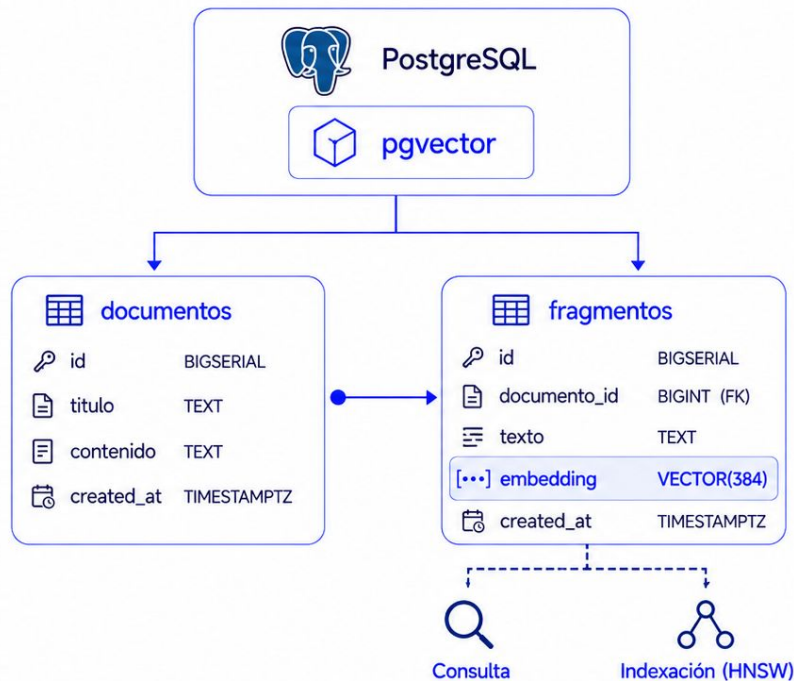
Enlace a la encuesta



Muchas gracias ¿Dudas?

Práctica

Modelar, almacenar, consultar e indexar fragmentos



Vamos a trabajar con PostgreSQL y pgvector para modelar, cargar, consultar e indexar fragmentos.

Práctica

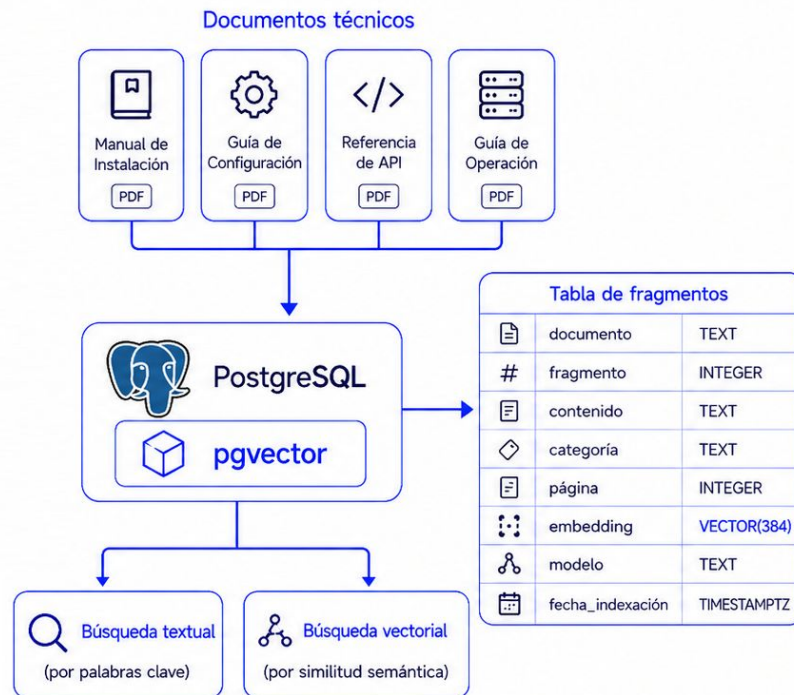
PostgreSQL y pgvector

Vamos a construir una base para almacenar fragmentos de documentación técnica.

Cada registro tendrá:

- documento de origen;
- número de fragmento;
- contenido;
- categoría;
- página;
- embedding;
- modelo utilizado;
- fecha de indexación.

Vamos a comparar búsqueda textual y búsqueda vectorial.



La práctica se concentra en cómo la base modela, almacena e indexa los vectores.

Práctica

Habilitar **pgvector** y crear las tablas

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE TABLE documentos (  
  id BIGSERIAL PRIMARY KEY,  
  titulo TEXT NOT NULL,  
  categoria TEXT,  
  activo BOOLEAN DEFAULT TRUE  
);
```

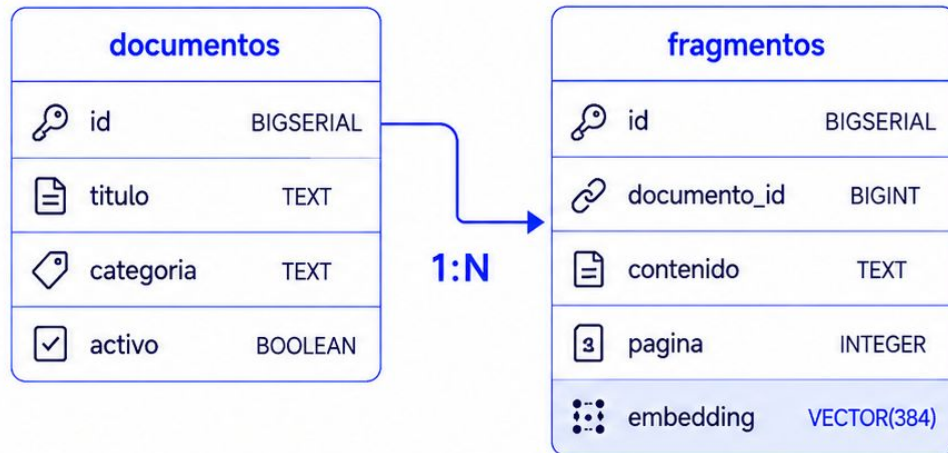
```
CREATE TABLE fragmentos (  
  id BIGSERIAL PRIMARY KEY,  
  documento_id BIGINT NOT NULL  
    REFERENCES documentos(id),  
  contenido TEXT NOT NULL,  
  pagina INTEGER,  
  embedding VECTOR(384)  
);
```



PostgreSQL



pgvector



Primero definimos la estructura relacional. Después incorporamos los datos vectoriales.

Práctica

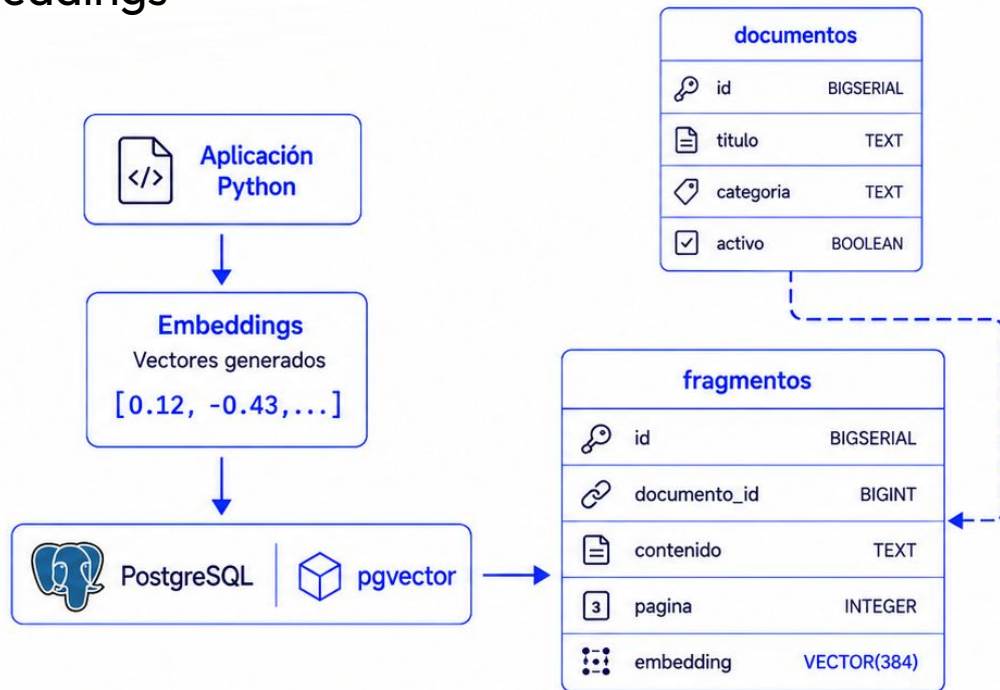
Insertar contenido, metadatos y embeddings

Vamos a:

- insertar documentos;
- crear fragmentos asociados;
- recibir embeddings generados desde Python;
- almacenar los vectores;
- validar que la dimensión sea correcta;
- revisar las relaciones.

Ejemplo conceptual:

```
INSERT INTO fragmentos (  
    documento_id,  
    contenido,  
    pagina,  
    embedding  
)  
VALUES (  
    1,  
    'Para restablecer la contraseña...',  
    4,  
    '[0.12, -0.43, ...]'  
);
```



Cada vector debe quedar vinculado con su contenido y documento de origen.

Práctica

Ordenar fragmentos por distancia

Consulta SQL

```
SELECT
  f.id,
  f.contenido,
  f.pagina,
  d.titulo,
  f.embedding <=> :vector_consulta
AS distancia
FROM fragmentos f
JOIN documentos d
  ON d.id = f.documento_id
WHERE d.activo = TRUE
ORDER BY distancia
LIMIT 5;
```



Vector de consulta
[0.12, -0.43,...]



PostgreSQL



pgvector



Búsqueda vectorial
(ORDER BY distancia)

Resultados ordenados por distancia (ascendente)

#	id (fragmento)	Contenido (fragmento)	Página	Documento (título)	Distancia
1	f_10234	La contraseña debe tener al menos 12 caracteres e incluir mayúsculas...	4	 Política de Seguridad	0.081
2	f_10087	No compartas tus credenciales con terceros bajo ninguna circunstancia.	2	 Política de Seguridad	0.127
3	f_11205	El acceso debe revocarse inmediatamente cuando...	7	 Gestión de Accesos	0.190
4	f_10122	Usa autenticación multifactor (MFA) siempre que sea posible.	5	 Política de Seguridad	0.241
5	f_11377	Los registros de auditoría deben conservarse por al menos 1 año.	9	 Cumplimiento Normativo	0.318



JOIN

Une fragmentos con
documentos (d.id = f.documento_id)



Filtro

Solo documentos
activos (activo = TRUE)



Cálculo de distancia

Usa el operador <=>
de pgvector



ORDER BY

Ordena por distancia
ascendente



LIMIT 5

Devuelve los 5
mejores resultados



La consulta combina JOIN, filtro, cálculo de distancia, ordenamiento y límite.

Práctica

Crear HNSW y comparar planes



CREATE INDEX

```
CREATE INDEX  
idx_fragmentos_embedding_hnsw  
ON fragmentos  
USING hnsw (  
    embedding vector_cosine_ops  
);
```



EXPLAIN ANALYZE

```
EXPLAIN ANALYZE  
SELECT  
    id,  
    contenido  
FROM fragmentos  
ORDER BY embedding <=> :vector_consulta  
LIMIT 5;
```

Vamos a comparar:



plan sin índice;



plan con índice;



tiempo de
ejecución;



filas
procesadas;



uso de
memoria;



calidad de los
resultados.

Comparación de planes de ejecución

Plan sin índice



Seq Scan

Recorrido completo de tabla



Filas procesadas $\approx 1.250.000$



Tiempo de ejecución ≈ 850 ms



Trabajo realizado **Alto**



Acceso a datos **Secuencial**



Plan con índice HNSW



Index Scan (HNSW)

Búsqueda por proximidad



Filas procesadas ≈ 500



Tiempo de ejecución ≈ 12 ms



Trabajo realizado **Bajo**



Acceso a datos **Rápido (índice)**

Más trabajo
Más filas
Más tiempo



Menos trabajo
Menos filas
Menos tiempo



Optimizar una consulta vectorial también requiere medir antes y después.